Python によるデータ分析入門

経済安全保障データ分析

全 12 回相当研修教材

（無償公開用 参考ひな形）

～pandas・seaborn・networkx で学ぶ実用データサイエンス～

【想定受講者】Excel 利用経験のみの方

プログラミング未経験でも大丈夫。

Google Colaboratory でブラウザだけで完結します。

著者: Étale Cohomology

2026 年 4 月公開

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

はじめに — なぜいま、Python と Excel の両方なのか

本講座は、Excel しか使ったことがないビジネスパーソンの方が、Python というプログラミング言語を使ってデータ分析の世界に踏み出すための入門書です。

「プログラミング」と聞くと身構えてしまうかもしれません。けれど、安心してください。本講座は次のような方のために書かれています。

- Excel で関数(VLOOKUP、SUMIF、IF など)を使ったことはある
- でも、プログラミング言語は触ったことがない
- 統計学やデータ分析の専門教育を受けたことがない
- 経済安全保障・地政学リスク・サプライチェーンに業務で関わっている
- 制裁対応・取引先審査・与信判断などで、データを使った判断を求められる

こうした方々に、最短ルートで「Python でデータを扱う感覚」を身につけていただくのが本講座の目的です。

なぜ Excel だけでは限界があるのか

Excel は素晴らしいツールです。多くの業務は Excel で十分に処理できます。しかし、次のような状況になると、Excel では対応が難しくなります。

> **Excel の限界が見えてくる場面**
>
> ・取引先リストが 10 万件を超え、Excel が重くて開けない
>
> ・OFAC 制裁リスト(12,000 件超、毎月数百件更新)を毎月手動でチェックしている
>
> ・複数の異なるデータソース(社内 DB、規制リスト、貿易統計)を結合したい
>
> ・同じ分析を毎月繰り返す必要があり、手作業に何時間もかけている
>
> ・グラフをきれいにカスタマイズしたいが、Excel の機能では物足りない
>
> ・地図上にデータをプロットしたい、ネットワーク図を描きたい

こうした場面で、Python というツールが力を発揮します。

なぜ Python なのか

Python は、世界中のデータサイエンティスト・研究者・エンジニアが使っているプログラミング言語です。次のような特徴があります。

- 無料で使える(Microsoft Office のような購入は不要)
- コードが英語の文章のように読みやすい
- データ分析・統計・可視化のための優れた道具(モジュール)が揃っている
- Google Colaboratory というブラウザ上で動く環境を使えば、PC にインストールせずに始められる
- 一度書いたコードを再利用すれば、毎月の定型作業が自動化できる

本講座では、Python の中でも特に重要な 4 つの道具(モジュール)を扱います。

> **本講座で学ぶ 4 つの Python モジュール**
>
> ① pandas(パンダス): Excel の表のようにデータを扱う道具
>
> ② seaborn(シーボーン): 美しいグラフを描く道具
>
> ③ networkx(ネットワークエックス): 関係の網(ネットワーク)を描く道具
>
> ④ matplotlib(マットプロットリブ): グラフ描画の基本道具(seaborn の土台)

Tableau 版との関係

本講座は、すでに公開されている「データサイエンティスト初級基礎講座 Tableau 活用 全 12 回」の Python 版です。同じデータセット・同じ経済安全保障の文脈で、Tableau をコードを書かずに使う方法ではなく、Python でコードを書いて分析する方法を学びます。

> **どちらから先に学ぶべきか**
>
> ・Tableau から始める: ドラッグ&ドロップでまず可視化に慣れたい方におすすめ
>
> ・Python から始める(本講座): 自動化・繰り返しの分析を見据えている方におすすめ

> ・両方学ぶ: 業務で重宝されるデータ分析者になります。多くの実務家が両方使っています

本講座の特徴

- Google Colaboratory に「コピー&ペースト」するだけですぐ動く完全版コード
- 各コードを 1 行ずつ丁寧に解説
- 使用するデータはすべて、無料で教育目的に使える公開オープンデータ
- 経済安全保障という実務文脈で学ぶので、業務にすぐ転用できる
- プログラミング未経験者を意識した、専門用語の言い換えと丁寧な説明

データ利用について

> **使用データのライセンス**
>
> 本講座で扱うデータは、すべてインターネット上で無料公開されているオープンデータです。
>
> 本教材は教育目的での参照を前提としており、データ自体の再配布は行いません。
>
> 受講者は演習の指示に従って各データソースの公式サイトから最新版をダウンロードしてください。
>
> 代表的なライセンス:
>
> ・米国政府公開データ(OFAC SDN, Entity List, CISA KEV など): 自由利用可能
>
> ・World Bank WGI: Creative Commons Attribution 4.0
>
> ・OECD Country Risk: 教育目的での参考使用(再配布不可)
>
> ・財務省貿易統計: 公開統計として一般利用可能
>
> 業務で実利用される際は、各データソースの最新の利用規約を必ずご確認ください。

目次

前半編(基礎・入門)

第 1 回 Google Colaboratory と Python のはじめの一歩

第 2 回 pandas でデータを読み込む — Excel からの卒業

第 3 回 データの前処理と加工 — pandas でクリーニング

第 4 回 データの分布を可視化する — seaborn で美しく

第 5 回 記述統計 — 平均・分散・相関を Python で

第 6 回 データ集計と前半編まとめ

後半編(発展・応用)

第 7 回 ネットワーク分析 — networkx で関係性を描く

第 8 回 時系列解析 — 地政学イベントの影響を見る

第 9 回 多変量解析 — カントリーリスクの複合評価

第 10 回 サイバーセキュリティデータの分析

第 11 回 統合分析レポート — 複数データを組み合わせる

第 12 回 総合演習と次へのステップ

付録

付録 A 公開データソース一覧

付録 B Python コードクイックリファレンス

付録 C よくあるエラーと対処法

第 1 回 Google Colaboratory と Python はじめの一歩

所要時間: 90 分 (環境準備 60 分 + ミニ演習 30 分)

1.1 Google Colaboratory とは何か

Google Colaboratory(以下「Colab」コラブ)は、Google が提供する無料の Python 実行環境です。次のような利点があります。

- PC に Python をインストールする必要がない
- ブラウザだけで完結する(Chrome、Edge、Firefox など)
- Google アカウントがあればすぐ使える
- 作成したプログラムは Google Drive 上に自動保存される
- 無料枠でもデータ分析の学習には十分

> **Excel と Colab の対応関係**
>
> Excel(エクセル)で言うと、
>
> ・Excel ファイル(.xlsx) ⇔ Colab のノートブック(.ipynb)
>
> ・Excel シート ⇔ Colab のセル(コードを書く欄)
>
> ・Excel の「実行」ボタン ⇔ Colab の「再生」ボタン(▶)
>
> ・Excel の計算結果 ⇔ Colab のセル下の出力
>
> という対応です。馴染みのある感覚で進められます。

1.2 Colab の起動と最初のコード

では、実際に Colab を使ってみましょう。

ステップ 1: Colab にアクセスする

ブラウザで次の URL にアクセスしてください。

https://colab.research.google.com/

Google アカウントでログインし、「ノートブックを新規作成」をクリックします。

ステップ 2: 最初のコードを書く

画面の真ん中に、コードを書く欄(セル)が表示されます。次のコードを入力してください。

```
# 最初の Python コード
print("こんにちは、Python!")
print("これからデータ分析を学びます")


# 簡単な計算もできる
x = 10
y = 20
print(f"{x} + {y} = {x + y}")
```

入力したら、セルの左にある▶(再生)ボタンを押すか、キーボードで Shift + Enter を押します。

セルの下に次のような結果が表示されれば成功です。

```
こんにちは、Python!
これからデータ分析を学びます
10 + 20 = 30
```

おめでとうございます

あなたはたった今、人生で初めての Python コードを実行しました。

難しいことは何もありません。Excel で関数を書くのと同じ感覚で、

Python でもコードを書いて、結果を確認することができます。

1.3 Python の基本ルール — 5 つだけ覚えてください

Python には様々なルールがありますが、本講座を進めるために必要なのは、たった 5 つの基本ルールだけです。

ルール 1: # は「コメント」を意味する

で始まる行は、Python からは無視されます。これは「コメント」と呼ばれ、自分やほかの人がコードを読んでわかりやすくするためのメモとして使います。

```
# このような行はコメントです(実行されません)
```

```
x = 100  # 行の途中からコメントを書くこともできます
print(x)  # 結果として 100 が表示されます
```

Excel との対応

Excel で言うと、セルに緑色の「セルにコメントを追加」と同じ感覚です。

計算には影響せず、説明だけを書く場所です。

ルール 2: 変数 = 値 で「名前と値」を結びつける

変数(へんすう)とは、値に「名前」をつけて記憶させる仕組みです。

```
# 変数に値を保存する
country_name = "日本"
sanction_count = 350
risk_score = 4.5


# 変数を使って計算する
total = sanction_count * 2
print(total)  # 結果: 700
```

Excel との対応

Excel で言うと、「セルに名前をつける」機能(=NAME 関数)と似ています。

A1 のセルに「sanction_count」という名前をつけて、

他のセルで =sanction_count * 2 と書けるイメージです。

ルール 3: 文字列は " または ' で囲む

文字を扱うときは、ダブルクォート " またはシングルクォート ' で囲みます。

```
# 文字列の例
country = "日本"
program = 'RUSSIA-EO14024'


# 数値と文字列は別物
age = 30          # 数値(クォートなし)
age_str = "30"    # 文字列(クォートあり)
```

ルール 4: 関数は 関数名(引数) で呼び出す

関数(かんすう)とは、ある処理を実行する命令のかたまりです。Excel の =SUM(A1:A10)

のような感覚です。

```
# print 関数: 画面に表示する
print("こんにちは")
```

```
# len 関数: 文字数を数える
text = "Étale Cohomology"
length = len(text)
print(length)  # 結果: 16


# round 関数: 数値を四捨五入する
score = 7.345
rounded = round(score, 2)  # 小数第 2 位まで
print(rounded)  # 結果: 7.35
```

Excel との対応

Excel の =SUM(範囲)、=COUNT(範囲)、=ROUND(数値, 桁) などと

ほぼ同じ感覚です。関数名と、カッコの中の引数(ひきすう)を

指定すると、結果が返ってきます。

ルール 5: インデント(字下げ)に意味がある

Python では、行の頭の「字下げ(インデント)」に重要な意味があります。これだけは Excel と違う部分なので注意してください。

```
# 条件によって処理を変える例
risk_score = 7


if risk_score >= 7:
    print("高リスクです")     # ← この行は 4 文字字下げ
    print("要注意!")          # ← この行も 4 文字字下げ
else:
    print("通常です")         # ← この行も 4 文字字下げ


# 字下げが揃っていないとエラーになる
```

⚠ インデントは必ず半角スペース 4 個で揃える

Python の作法では、字下げは「半角スペース 4 個」で統一するのが標準です。

Tab キーで字下げしても見た目は揃いますが、

スペースと Tab を混在させるとエラーが出るので、

Colab が自動でやってくれる字下げに任せるのが安全です。

1.4 ミニ演習: SDN 件数の簡単な集計

最後に、本格的な分析の前に「Python で数を集計する」感覚を体験してみましょう。

以下のコードを Colab にコピー&ペーストして実行してください。

```
# 演習 1-1: SDN 登録件数を国別にカウントする(練習データ)
# このコードをまるごとコピーして Colab のセルに貼り付け、実行してください


# サンプルデータ(国別の制裁登録件数のメモ)
sanction_data = {
    "Russia": 1850,
    "Iran": 980,
    "North Korea": 420,
    "Cuba": 210,
    "Syria": 340,
    "Venezuela": 280,
    "Myanmar": 120,
    "Belarus": 380
}


# データを表示する
print("【国別 SDN 登録件数】")
for country, count in sanction_data.items():
    print(f"  {country}: {count}件")


# 合計を計算する
total = sum(sanction_data.values())
print(f"\n 合計: {total}件")


# 最も多い国を見つける
max_country = max(sanction_data, key=sanction_data.get)
print(f"最も多い国: {max_country} ({sanction_data[max_country]}件)")


# 平均を計算する
average = total / len(sanction_data)
print(f"平均: {average:.1f}件")
```

実行すると、次のような結果が表示されます。

```
【国別 SDN 登録件数】
  Russia: 1850 件
  Iran: 980 件
  North Korea: 420 件
  Cuba: 210 件
  Syria: 340 件
  Venezuela: 280 件
  Myanmar: 120 件
  Belarus: 380 件


合計: 4580 件
最も多い国: Russia (1850 件)
平均: 572.5 件
```

コードを読み解いてみましょう

> ・sanction_data = {...} は「国名と件数のペア」を保存する辞書(dictionary)
>
> ・for country, count in sanction_data.items(): は「すべての国を順に取り出す」
>
> ・sum() は合計、max() は最大、len() は個数を数える関数
>
> ・f"..." の形は「f 文字列」と呼ばれ、変数を文字列に埋め込めます
>
> 最初は「呪文」に見えるかもしれませんが、繰り返し使ううちに自然になじみます。
>
> Excel の関数を覚えたときと同じです。

1.5 第 1 回のまとめ

- Google Colaboratory はブラウザだけで Python が使える無料環境
- Python の基本ルールは「コメント、変数、文字列、関数、インデント」の 5 つ
- Python は Excel の関数と似た感覚で書ける
- 簡単なデータ集計なら、初日から Python で実行できる

次回は、いよいよ pandas というモジュールを使って、本格的なデータ分析の世界に入っていきます。

💻 第 1 回 実行結果例

▼ Section 1.2 (First code)

```
こんにちは、Python!
これからデータ分析を学びます
10 + 20 = 30
```

▼ Section 1.3 Rule1 (Comment)

```
100
```

▼ Section 1.3 Rule2 (Variable)

```
700
```

▼ Section 1.3 Rule4 (Function)

```
こんにちは
16
7.34
```

▼ Section 1.3 Rule5 (Indent)

```
高リスクです
要注意!
```

▼ Section 1.4 Mini Exercise

```
【国別 SDN 登録件数】
  Russia: 1850 件
  Iran: 980 件
  North Korea: 420 件
  Cuba: 210 件
  Syria: 340 件
  Venezuela: 280 件
  Myanmar: 120 件
  Belarus: 380 件

合計: 4580 件
最も多い国: Russia (1850 件)
平均: 572.5 件
```

第 2 回 pandas でデータを読み込む — Excel からの卒業

所要時間: 120 分 (講義 60 分 + 演習 60 分)

2.1 pandas とは何か

pandas(パンダス)は、Python でデータを扱うときに最もよく使われるモジュールです。一言で言えば、

pandas の役割

「Excel の表を Python で操作するためのモジュール」

pandas を使えば、

・CSV ファイル、Excel ファイル、Web からデータを読み込める

・読み込んだ表を「DataFrame(データフレーム)」という Excel 風の表として扱える

・行や列の選択、フィルタリング、並べ替え、集計などがコード 1 行でできる

・処理結果を CSV や Excel として書き出せる

2.2 DataFrame — Python の「Excel シート」

pandas で扱う表は DataFrame(データフレーム)と呼ばれます。これは Excel シートとほぼ同じ概念です。

Excel の用語	pandas の用語	意味
シート	DataFrame	表全体
行	行(row)	1 件のデータ
列	列(column)、Series	1 つの項目
セル	セル(cell)	1 つの値
見出し行	カラム名(column name)	各列の名前
行番号(1, 2, 3...)	インデックス(index)	各行の番号や名前

2.3 pandas のインストールと読み込み

Google Colab では、pandas は最初からインストール済みです。使うときは、コードの最初に 1 行書くだけです。

```
# pandas を読み込む(おまじない)
import pandas as pd
```

import pd as pd の意味

> ・import pandas: 「pandas モジュールを使う準備をする」
>
> ・as pd: 「これからは pd という短い名前で呼ぶ」
>
>
>
> 毎回 pandas.read_csv() と書くと長いので、
>
> pd.read_csv() と短く書けるようにしているだけです。
>
> 世界中の Python ユーザーがこの「pd」という略称を使うのが慣習です。

2.4 演習 2-1: Web から CSV を読み込む

では、実際に経済安全保障のデータを読み込んでみましょう。

ここでは、CISA(米国サイバーセキュリティ庁)が公開している「悪用された脆弱性カタログ(KEV)」の CSV を直接 Web から読み込みます。

```
# 演習 2-1: CISA KEV(悪用された脆弱性)を読み込む
# このコードをそのまま Colab に貼り付けて実行してください


import pandas as pd


# CISA の公式 CSV を直接 Web から読み込む
url =
"https://www.cisa.gov/sites/default/files/csv/known_exploited_vulnerabilities.csv"
df = pd.read_csv(url)


# 最初の 5 行を表示
print("【データの最初の 5 行】")
print(df.head())


# データの大きさ(行数、列数)を表示
print(f"\n 行数: {df.shape[0]}")
print(f"列数: {df.shape[1]}")


# 列名の一覧を表示
print(f"\n 列名: {list(df.columns)}")
```

実行すると、最新の脆弱性データが DataFrame として読み込まれ、最初の 5 行と全体のサイズが表示されます。

> **驚くべきこと**

> たった 3 行のコード(import、url、pd.read_csv)で、
>
> Web 上の最新 CSV を読み込めました。
>
>
>
> Excel で毎月「ダウンロード→開く→必要部分をコピー」していた作業が、
>
> Python では「URL を指定するだけ」で完了します。
>
> しかも、データが更新されたら、同じコードを再実行するだけで最新版に更新されます。

2.5 DataFrame の基本操作

DataFrame を読み込んだら、Excel で「データの中身を見る」「特定の列だけ抽出する」「条件で絞り込む」といった操作をしたくなります。pandas ではすべて 1 行のコードで実行できます。

基本操作 1: データの概要を見る

```
# データの最初の 5 行を見る(Excel の「先頭」を見る感覚)
df.head()


# 最後の 5 行を見る
df.tail()


# 行数・列数を見る
df.shape


# 列名一覧を見る
df.columns


# 各列のデータ型と欠損数を見る
df.info()


# 数値列の統計量(平均、標準偏差、最大、最小など)を見る
df.describe()
```

基本操作 2: 特定の列を抽出する

```
# 1 つの列を抽出する(Excel で言うと「列を選択」)
df['vendorProject']
```

```
# 複数の列を抽出する
df[['vendorProject', 'product', 'vulnerabilityName']]
```

[ ] と [[ ]] の違い

・df['列名'] → Series(1 列だけのデータ)が返る

・df[['列名 1', '列名 2']] → DataFrame(複数列の表)が返る

Excel で「セル 1 個」と「セル範囲」が違うのと同じ感覚です。

基本操作 3: 条件で絞り込む(Excel のオートフィルター)

```
# Microsoft 社の脆弱性だけを抽出する
microsoft_df = df[df['vendorProject'] == 'Microsoft']
print(microsoft_df.head())


# 件数を表示
print(f"Microsoft 関連の脆弱性件数: {len(microsoft_df)}")


# 複数条件: Microsoft かつ Windows という名前を含む
windows_df = df[(df['vendorProject'] == 'Microsoft') &
                (df['product'].str.contains('Windows', na=False))]
print(f"Microsoft Windows 関連: {len(windows_df)}件")
```

⚠ 複数条件のときの注意

・Python では「かつ」は & 、「または」は | を使います

・各条件はそれぞれ ( ) で囲みます

・&& や || は使えません(他のプログラミング言語で使うものです)

間違いやすいので、最初は「条件を ( ) で囲んで & でつなぐ」と覚えてください。

基本操作 4: 並べ替える(Excel の「並べ替え」)

```
# CVSS スコア(脆弱性の深刻度)が高い順に並べる
df_sorted = df.sort_values('cvssV3_baseScore', ascending=False)
print(df_sorted[['vendorProject', 'product', 'cvssV3_baseScore']].head(10))
```

基本操作 5: 集計する(Excel のピボットテーブル)

```
# ベンダー別の脆弱性件数を集計する
vendor_counts = df.groupby('vendorProject').size().sort_values(ascending=False)
print("【ベンダー別 脆弱性件数 トップ 10】")
print(vendor_counts.head(10))
```

groupby() はピボットテーブルの神髄

> df.groupby('列名').size() という書き方は、
>
> Excel のピボットテーブルで「行に列名を置いて、件数を集計する」
>
> という操作と同じです。
>
>
> groupby('列名').sum() → 合計
>
> groupby('列名').mean() → 平均
>
> groupby('列名').max() → 最大値
>
>
> ピボットテーブルでマウス操作していた集計が、コード 1 行で実行できます。

2.6 演習 2-2: WGI(世界ガバナンス指標)を読み込む

もう 1 つ、本格的なデータ分析の例として、World Bank が公開している WGI(世界ガバナンス指標)を読み込んでみましょう。

```
# 演習 2-2: World Bank WGI を Web から読み込む
# このコードを Colab に貼り付けて実行してください


import pandas as pd


# World Bank Open Data API から腐敗認識スコア(CC)を取得
# (国別、年別の腐敗管理指標)
url =
"https://api.worldbank.org/v2/country/all/indicator/CC.EST?date=2020:2023&format=js
on&per_page=2000"


# JSON として取得
import requests
response = requests.get(url)
data = response.json()


# 2 番目の要素にデータが入っている
records = data[1]


# DataFrame に変換
df_wgi = pd.DataFrame(records)


# 必要な列だけ抽出して整形
```

```
df_clean = df_wgi[['country', 'date', 'value']].copy()
df_clean['country_name'] = df_clean['country'].apply(lambda x: x['value'])
df_clean = df_clean[['country_name', 'date', 'value']]
df_clean = df_clean.dropna()  # 欠損値を除外


# 結果を表示
print("【World Bank WGI - Control of Corruption】")
print(df_clean.head(10))
print(f"\n データ件数: {len(df_clean)}件")
print(f"対象期間: {df_clean['date'].min()} - {df_clean['date'].max()}")
```

🎯 API ってなに?

API(エーピーアイ)は、Web サービスがデータを機械的に提供するための窓口です。

World Bank、OECD、CISA など、多くの公的機関が API を公開しています。

API を使うと、データを「直接」「最新版で」「自動で」取得できるので、

毎月手動で CSV をダウンロードする手間がなくなります。

本講座では、難しい API の仕組みは深入りせず、

「URL を指定して読み込む」レベルの使い方だけ扱います。

2.7 第 2 回のまとめ

- pandas は Python の「Excel 代わり」のモジュール
- DataFrame は Excel シートに相当する表のかたまり
- pd.read_csv() で Web から直接データを読み込める
- head(), info(), describe() で素早くデータを把握できる
- 条件絞り込み・並べ替え・集計がコード 1 行で実行できる

第 2 回 実行結果例

⚠ 以下の実行結果は、架空データを用いた結果です。受講生の皆様はご自分で実際にウェブサイトからオープンデータを取得した上で、pandas を実行してください。

▼ Section 2.4 (CISA KEV Sample)

```
【データの最初の 5 行】
            cveID vendorProject  ... knownRansomwareCampaignUse cvssV3_baseScore
0  CVE-2021-44228        Apache  ...                      Known             10.0
1  CVE-2021-26855     Microsoft  ...                      Known              9.1
2  CVE-2022-30190     Microsoft  ...                      Known              7.8
3  CVE-2023-23397     Microsoft  ...                    Unknown              9.8
4  CVE-2021-34527     Microsoft  ...                      Known              8.8

[5 rows x 10 columns]

行数: 10
列数: 10

列名: ['cveID', 'vendorProject', 'product', 'vulnerabilityName', 'dateAdded',
'shortDescription', 'requiredAction', 'dueDate', 'knownRansomwareCampaignUse',
'cvssV3_baseScore']
```

▼ Section 2.5 Basic1

```
--- df.head() ---
            cveID vendorProject  ... knownRansomwareCampaignUse cvssV3_baseScore
0  CVE-2021-44228        Apache  ...                      Known             10.0
1  CVE-2021-26855     Microsoft  ...                      Known              9.1
2  CVE-2022-30190     Microsoft  ...                      Known              7.8
3  CVE-2023-23397     Microsoft  ...                    Unknown              9.8
4  CVE-2021-34527     Microsoft  ...                      Known              8.8

[5 rows x 10 columns]

--- df.tail() ---
            cveID vendorProject  ... knownRansomwareCampaignUse cvssV3_baseScore
5   CVE-2020-1472     Microsoft  ...                      Known             10.0
6  CVE-2022-22965        VMware  ...                    Unknown              9.8
7  CVE-2017-11882     Microsoft  ...                      Known              7.8
8   CVE-2023-3519        Citrix  ...                      Known              9.4
9  CVE-2021-40444     Microsoft  ...                    Unknown              8.8

[5 rows x 10 columns]

--- df.shape ---
(10, 10)

--- df.columns ---
['cveID', 'vendorProject', 'product', 'vulnerabilityName', 'dateAdded',
'shortDescription', 'requiredAction', 'dueDate', 'knownRansomwareCampaignUse',
'cvssV3_baseScore']
```

▼ Section 2.5 df.info()

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 10 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   cveID                       10 non-null     str
```

```
 1   vendorProject               10 non-null     str
 2   product                     10 non-null     str
 3   vulnerabilityName           10 non-null     str
 4   dateAdded                   10 non-null     str
 5   shortDescription            10 non-null     str
 6   requiredAction              10 non-null     str
 7   dueDate                     10 non-null     str
 8   knownRansomwareCampaignUse  10 non-null     str
 9   cvssV3_baseScore            10 non-null     float64
dtypes: float64(1), str(9)
memory usage: 932.0 bytes
```

▼ Section 2.5 df.describe()

```
       cvssV3_baseScore
count         10.000000
mean           9.130000
std            0.832733
min            7.800000
25%            8.800000
50%            9.250000
75%            9.800000
max           10.000000
```

▼ Section 2.5 Basic2 (Column extract)

```
--- df['vendorProject'] ---
0       Apache
1    Microsoft
2    Microsoft
3    Microsoft
4    Microsoft
5    Microsoft
6       VMware
7    Microsoft
8       Citrix
9    Microsoft
Name: vendorProject, dtype: str

--- df[['vendorProject','product','vulnerabilityName']] ---
  vendorProject           product             vulnerabilityName
0        Apache            Log4j2                 Log4Shell RCE
1     Microsoft   Exchange Server                    ProxyLogon
2     Microsoft           Windows                   Follina MSDT
3     Microsoft           Outlook  Outlook Privilege Escalation
4     Microsoft           Windows                PrintNightmare
5     Microsoft           Windows                     Zerologon
6        VMware  Spring Framework                  Spring4Shell
7     Microsoft            Office      Office Memory Corruption
8        Citrix     NetScaler ADC                  Citrix Bleed
9     Microsoft           Windows                    MSHTML RCE
```

▼ Section 2.5 Basic3 (Filter)

```
            cveID vendorProject  ... knownRansomwareCampaignUse cvssV3_baseScore
1  CVE-2021-26855     Microsoft  ...                      Known              9.1
2  CVE-2022-30190     Microsoft  ...                      Known              7.8
3  CVE-2023-23397     Microsoft  ...                    Unknown              9.8
4  CVE-2021-34527     Microsoft  ...                      Known              8.8
5   CVE-2020-1472     Microsoft  ...                      Known             10.0

[5 rows x 10 columns]
Microsoft 関連の脆弱性件数: 7
Microsoft Windows 関連: 4 件
```

▼ Section 2.5 Basic4 (Sort)

```
  vendorProject           product  cvssV3_baseScore
0        Apache            Log4j2              10.0
5     Microsoft           Windows              10.0
6        VMware  Spring Framework               9.8
3     Microsoft           Outlook               9.8
8        Citrix     NetScaler ADC               9.4
1     Microsoft   Exchange Server               9.1
9     Microsoft           Windows               8.8
4     Microsoft           Windows               8.8
2     Microsoft           Windows               7.8
7     Microsoft            Office               7.8
```

第 3 回 データの前処理と加工 — pandas でクリーニング

所要時間: 120 分

3.1 「データの前処理」とは何か

実務でデータを扱うとき、最初の状態のままで分析できることはほとんどありません。

> **実データには「汚れ」がある**
>
> ・欠損値(空白セル、NaN): 「不明」「未入力」などで値がない
>
> ・表記ゆれ: 「日本」「JAPAN」「Japan」が同じ国を指している
>
> ・余分なスペース: 「 ロシア 」のように前後に空白がある
>
> ・データ型の違い: 数字なのに文字列として保存されている
>
> ・重複行: 同じデータが 2 度入っている
>
> ・形式の違い: 日付が「2024/01/15」だったり「Jan 15, 2024」だったり
>
> こうした「汚れ」を取り除く作業を「データクリーニング」または
>
> 「データ前処理」と呼びます。実務では分析時間の 7-8 割がこの作業です。

3.2 欠損値(NaN)を扱う

Python では、空白セルは NaN(ナン、Not a Number の略)として扱われます。Excel でいう「空白セル」と同じ意味です。

欠損値を見つける

```
# 演習 3-1: 欠損値の確認と処理
import pandas as pd
import numpy as np


# サンプルデータ(取引先リストに欠損あり)
data = {
    'company': ['Alpha Corp', 'Beta Inc', 'Gamma Ltd', 'Delta Co', 'Epsilon
Group'],
    'country': ['Japan', 'USA', None, 'Russia', 'China'],
    'risk_score': [2.1, 3.5, np.nan, 7.8, 5.2],
    'sanctioned': [False, False, False, True, np.nan]
}
df = pd.DataFrame(data)
print("【元のデータ】")
print(df)


# 欠損値の数を列ごとに数える
print("\n【欠損値の数】")
print(df.isnull().sum())


# 欠損値を含む行を表示
print("\n【欠損値を含む行】")
print(df[df.isnull().any(axis=1)])
```

欠損値を処理する

```
# 方法 1: 欠損値を含む行を削除する
df_dropped = df.dropna()
print("【欠損行を削除した結果】")
print(df_dropped)


# 方法 2: 欠損値を埋める(代入する)
df_filled = df.copy()
df_filled['country'] = df_filled['country'].fillna('Unknown')
df_filled['risk_score'] =
df_filled['risk_score'].fillna(df_filled['risk_score'].mean())
df_filled['sanctioned'] = df_filled['sanctioned'].fillna(False)
print("\n【欠損値を埋めた結果】")
print(df_filled)
```

🎯 欠損値を「削除」と「埋める」のどちらにすべきか

・データが少ない/欠損が少ない → 削除しても問題ない

・分析対象の重要な列が欠損 → その行は削除

・欠損が多い列 → 平均値、中央値、または「Unknown」で埋める

・お金や件数 → 0 で埋めても良い場合がある

> ・絶対の正解はありません。「分析の目的」に応じて判断します。

3.3 表記ゆれを統一する

「日本」「JAPAN」「Japan」「日 本」のように、同じ意味でも書き方がバラバラなことを「表記ゆれ」と言います。

```
# 演習 3-2: 表記ゆれを統一する
import pandas as pd


# サンプルデータ(国名の表記がバラバラ)
data = {
    'company': ['A 社', 'B 社', 'C 社', 'D 社', 'E 社', 'F 社'],
    'country': ['日本', 'JAPAN', ' japan ', 'Japan', '日 本', 'JPN']
}
df = pd.DataFrame(data)
print("【元のデータ】")
print(df)


# ステップ 1: 前後の空白を削除
df['country'] = df['country'].str.strip()


# ステップ 2: 大文字に統一
df['country'] = df['country'].str.upper()


# ステップ 3: 空白を削除(全角空白も含む)
df['country'] = df['country'].str.replace(' ', '').str.replace('　', '')


# ステップ 4: 表記ゆれを置換(変換辞書を使う)
country_mapping = {
    'JAPAN': 'JP',
    '日本': 'JP',
    'JPN': 'JP'
}
df['country_code'] = df['country'].replace(country_mapping)


print("\n【表記統一後】")
print(df)
```

3.4 データ型の変換

CSV から読み込んだデータでは、本来「数値」のはずの列が「文字列」として読み込まれることがあります。これを正しい型に変換します。

```
# 演習 3-3: データ型の変換
import pandas as pd


# 文字列として保存されている数値
data = {
    'company': ['A', 'B', 'C', 'D'],
    'revenue_str': ['1,000,000', '2,500,000', '750,000', '3,200,000'],
    'risk_score_str': ['2.5', '4.8', '7.2', '3.1'],
    'date_str': ['2024-01-15', '2024-02-20', '2024-03-10', '2024-04-05']
}
df = pd.DataFrame(data)


# 元のデータ型を確認
print("【元のデータ型】")
print(df.dtypes)


# 文字列を数値に変換(カンマも除去)
df['revenue'] = df['revenue_str'].str.replace(',', '').astype(int)
df['risk_score'] = df['risk_score_str'].astype(float)


# 文字列を日付に変換
df['date'] = pd.to_datetime(df['date_str'])


# 変換後のデータ型を確認
print("\n【変換後のデータ型】")
print(df.dtypes)
print("\n【変換後のデータ】")
print(df[['company', 'revenue', 'risk_score', 'date']])
```

3.5 新しい列を追加する(計算列)

Excel で「列を追加して数式で計算する」のと同じことを、pandas でも行います。

```
# 演習 3-4: 計算列の追加
import pandas as pd


data = {
    'country': ['Japan', 'USA', 'Germany', 'Russia', 'China'],
    'export_2023': [4500, 8200, 5100, 1200, 6800],
    'export_2024': [4700, 8500, 5300, 800, 7100],
    'risk_score': [2.1, 3.0, 2.8, 7.5, 5.5]
}
df = pd.DataFrame(data)


# 単純な計算列
df['change'] = df['export_2024'] - df['export_2023']
df['change_pct'] = (df['change'] / df['export_2023']) * 100
```

```
# 条件分岐(if-else)で列を作る(Excel の IF 関数と同じ感覚)
df['risk_level'] = df['risk_score'].apply(
    lambda x: '高' if x >= 7 else '中' if x >= 4 else '低'
)


# リスク調整後の輸出額
df['risk_adjusted_export'] = df['export_2024'] / (df['risk_score'] / 10 + 1)


print(df.round(2))
```

lambda(ラムダ)とは何か

lambda x: ... という書き方は「使い捨ての小さな関数」を作る方法です。

lambda x: '高' if x >= 7 else ('中' if x >= 4 else '低')

は「x を受け取って、7 以上なら'高'、4 以上なら'中'、それ以外なら'低'を返す関数」

Excel の IF 関数 のネストと同じ感覚です:

=IF(A1>=7, "高", IF(A1>=4, "中", "低"))

3.6 複数の表を結合する(VLOOKUP の代わり)

Excel で「VLOOKUP 関数」を使って 2 つの表を結合した経験はありませんか?pandas では、もっと簡単で強力な方法があります。

```
# 演習 3-5: 2 つの DataFrame を結合する(JOIN/MERGE)
import pandas as pd


# 表 1: 取引先リスト
companies = pd.DataFrame({
    'company_id': ['C001', 'C002', 'C003', 'C004', 'C005'],
    'name': ['Alpha', 'Beta', 'Gamma', 'Delta', 'Epsilon'],
    'country_code': ['JP', 'US', 'CN', 'RU', 'DE']
})


# 表 2: 国別カントリーリスクスコア
country_risk = pd.DataFrame({
    'country_code': ['JP', 'US', 'CN', 'RU', 'DE', 'IN'],
    'country_name': ['Japan', 'USA', 'China', 'Russia', 'Germany', 'India'],
    'risk_score': [2.1, 3.0, 5.5, 7.5, 2.8, 4.2]
})
```

```
# 結合(merge)する: country_code をキーにして
merged = companies.merge(country_risk, on='country_code', how='left')


print("【結合後の表】")
print(merged)


# Excel の =VLOOKUP(B2, country_risk!A:C, 2, FALSE) と同じこと
```

結合の種類

・how='left' → 左の表(companies)を全部残す。VLOOKUP と同じ動き

・how='right' → 右の表(country_risk)を全部残す

・how='inner' → 両方に存在するキーだけを残す

・how='outer' → 両方の表を全部残す

実務ではほとんど how='left' で十分です。

3.7 第 3 回のまとめ

- 実データには欠損値・表記ゆれ・型の違いなどの「汚れ」がある
- isnull(), dropna(), fillna() で欠損値を扱える
- .str.strip(), .str.upper() などで表記ゆれを統一
- astype() でデータ型を変換できる
- 計算列の追加は「df['新列'] = ...」と書くだけ
- merge() で VLOOKUP に代わる強力な結合ができる

第 3 回 実行結果例

▼ Exercise 3-1 (Missing values)

```
【元のデータ】
         company country  risk_score sanctioned
0     Alpha Corp   Japan         2.1      False
1       Beta Inc     USA         3.5      False
2      Gamma Ltd     NaN         NaN      False
3       Delta Co  Russia         7.8       True
4  Epsilon Group   China         5.2        NaN

【欠損値の数】
company       0
country       1
risk_score    1
sanctioned    1
dtype: int64

【欠損値を含む行】
         company country  risk_score sanctioned
2      Gamma Ltd     NaN         NaN      False
4  Epsilon Group   China         5.2        NaN

【欠損行を削除した結果】
      company country  risk_score sanctioned
0  Alpha Corp   Japan         2.1      False
1    Beta Inc     USA         3.5      False
3    Delta Co  Russia         7.8       True

【欠損値を埋めた結果】
         company  country  risk_score sanctioned
0     Alpha Corp    Japan        2.10      False
1       Beta Inc      USA        3.50      False
2      Gamma Ltd  Unknown        4.65      False
3       Delta Co   Russia        7.80       True
4  Epsilon Group    China        5.20      False
```

▼ Exercise 3-2 (Normalize text)

```
【元のデータ】
  company  country
0      A 社       日本
1      B 社    JAPAN
2      C 社   japan
3      D 社    Japan
4      E 社      日 本
5      F 社      JPN

【表記統一後】
  company country country_code
0      A 社      日本           JP
1      B 社   JAPAN           JP
2      C 社   JAPAN           JP
3      D 社   JAPAN           JP
4      E 社      日本           JP
5      F 社     JPN           JP
```

▼ Exercise 3-3 (Type conversion)

```
【元のデータ型】
company           str
revenue_str       str
```

```
risk_score_str    str
date_str          str
dtype: object

【変換後のデータ型】
company                      str
revenue_str                  str
risk_score_str               str
date_str                     str
revenue                    int64
risk_score               float64
date              datetime64[us]
dtype: object

【変換後のデータ】
  company  revenue  risk_score       date
0       A  1000000         2.5 2024-01-15
1       B  2500000         4.8 2024-02-20
2       C   750000         7.2 2024-03-10
3       D  3200000         3.1 2024-04-05
```

▼ Exercise 3-4 (Calculated columns)

```
   country  export_2023  ...  risk_level  risk_adjusted_export
0    Japan         4500  ...           低               3884.30
1      USA         8200  ...           低               6538.46
2  Germany         5100  ...           低               4140.62
3   Russia         1200  ...           高                457.14
4    China         6800  ...           中               4580.65

[5 rows x 8 columns]
```

▼ Exercise 3-5 (Merge)

```
【結合後の表】
  company_id     name country_code country_name  risk_score
0       C001    Alpha           JP        Japan         2.1
1       C002     Beta           US          USA         3.0
2       C003    Gamma           CN        China         5.5
3       C004    Delta           RU       Russia         7.5
4       C005  Epsilon           DE      Germany         2.8
```

第 4 回 データの分布を可視化する — seaborn で美しく

所要時間: 120 分

4.1 seaborn とは何か

seaborn(シーボーン)は、Python でグラフを描くためのモジュールです。matplotlib という基本モジュールの上に作られていて、より少ないコードで、より美しいグラフが描けます。

> **seaborn の長所**

> ・コード 3 行で論文レベルのグラフが描ける
>
> ・色合いがデフォルトで美しい
>
> ・統計的な分析機能(回帰直線、分布、エラーバー)が簡単に追加できる
>
> ・Excel のグラフより細かいカスタマイズが可能

4.2 seaborn の読み込み

```
# seaborn と関連モジュールの読み込み(おまじない)
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt


# 日本語の文字化け対策
plt.rcParams['font.family'] = 'DejaVu Sans'


# グラフのスタイル設定
sns.set_style("whitegrid")
sns.set_palette("husl")
```

4.3 演習 4-1: 箱ひげ図(Box Plot)でカントリーリスクの分布を見る

Tableau 編で学んだ Box Plot を、Python でも描いてみましょう。

```
# 演習 4-1: 地域別カントリーリスクの箱ひげ図
# このコードを Colab に貼り付けて実行してください


import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np


# サンプルデータ(地域別のカントリーリスクスコア)
np.random.seed(42)
data = {
    'region': ['Asia']*15 + ['Europe']*15 + ['Middle East']*15 + ['Africa']*15,
    'risk_score': (
        list(np.random.normal(3.5, 1.5, 15)) +
        list(np.random.normal(2.0, 1.0, 15)) +
        list(np.random.normal(6.0, 2.0, 15)) +
        list(np.random.normal(5.0, 2.0, 15))
    )
}
df = pd.DataFrame(data)


# 箱ひげ図を描く
```

```
plt.figure(figsize=(10, 6))
sns.boxplot(x='region', y='risk_score', data=df, palette='Set2')
plt.title('Country Risk Score Distribution by Region', fontsize=14)
plt.xlabel('Region', fontsize=12)
plt.ylabel('Risk Score (0-10)', fontsize=12)
plt.show()


# 各地域の統計量を表示
print("【地域別 統計量】")
print(df.groupby('region')['risk_score'].describe().round(2))
```

Box Plot の読み方(復習)

・箱の中の線 = 中央値(データを並べた真ん中の値)

・箱の下端 = 第 1 四分位数(下から 25%の位置)

・箱の上端 = 第 3 四分位数(下から 75%の位置)

・ひげ = 四分位範囲の 1.5 倍以内の最大・最小

・箱の外の点 = 外れ値(異常に高い・低い値)

4.4 演習 4-2: ヒストグラムで件数の分布を見る

```
# 演習 4-2: 制裁件数のヒストグラム
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np


# サンプルデータ(架空の 100 カ国の制裁件数)
np.random.seed(0)
sanctions = np.concatenate([
    np.random.poisson(20, 70),     # 多くの国は 20 件前後
    np.random.poisson(200, 25),    # 一部は 200 件前後
    np.random.poisson(1500, 5)     # 数か国だけ突出
])


df = pd.DataFrame({'sanction_count': sanctions})


# ヒストグラムを描く
plt.figure(figsize=(10, 6))
sns.histplot(data=df, x='sanction_count', bins=30, color='steelblue')
plt.title('Distribution of Sanction Counts (100 countries)', fontsize=14)
plt.xlabel('Number of Sanctions', fontsize=12)
plt.ylabel('Number of Countries', fontsize=12)
plt.axvline(df['sanction_count'].mean(), color='red', linestyle='--',
            label=f'Mean: {df["sanction_count"].mean():.0f}')
plt.axvline(df['sanction_count'].median(), color='green', linestyle='--',
            label=f'Median: {df["sanction_count"].median():.0f}')
```

```
plt.legend()
plt.show()


print(f"平均値: {df['sanction_count'].mean():.0f}件")
print(f"中央値: {df['sanction_count'].median():.0f}件")
print(f"差: {df['sanction_count'].mean() - df['sanction_count'].median():.0f}件")
```

平均と中央値が大きく違うとき

上のグラフでは、平均が中央値より大きく上にあります。

これは「ごく一部の国が極端に多くの制裁を受けている」ため、

平均が引っ張り上げられている状態です。

このような分布を「右に裾の長い分布(右に歪んでいる)」と呼びます。

経済安全保障では、こうした「少数の国に集中する」傾向がよく見られます。

判断材料として、平均値だけでなく中央値も見ることが大切です。

4.5 演習 4-3: 散布図で 2 つの変数の関係を見る

```
# 演習 4-3: カントリーリスクと貿易量の散布図
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np


# サンプルデータ
np.random.seed(123)
n_countries = 50
risk = np.random.uniform(1, 8, n_countries)
# リスクが高いほど貿易量は減る傾向(ただしばらつきあり)
trade = 10000 - 800 * risk + np.random.normal(0, 1500, n_countries)
trade = np.maximum(trade, 100)  # 最低でも 100


df = pd.DataFrame({
    'country_risk': risk,
    'trade_volume': trade
})


# 散布図 + 回帰直線を描く
plt.figure(figsize=(10, 6))
sns.regplot(x='country_risk', y='trade_volume', data=df,
            scatter_kws={'alpha': 0.6, 's': 60},
            line_kws={'color': 'red'})
```

```
plt.title('Country Risk vs Trade Volume', fontsize=14)
plt.xlabel('Country Risk Score', fontsize=12)
plt.ylabel('Trade Volume (million USD)', fontsize=12)
plt.show()


# 相関係数を計算する
correlation = df['country_risk'].corr(df['trade_volume'])
print(f"相関係数: {correlation:.3f}")
print(f"解釈: {'強い負の相関' if correlation < -0.5 else '弱い相関'}")
```

相関係数の見方

相関係数は -1 から +1 までの値を取ります。

・+1 に近い → 強い正の相関(片方が増えると、もう片方も増える)

・0 に近い → 相関がない(関係がない)

・-1 に近い → 強い負の相関(片方が増えると、もう片方は減る)

目安:

・|r| > 0.7 → 強い相関

・|r| 0.3-0.7 → 中程度の相関

・|r| < 0.3 → 弱い相関

ただし「相関がある = 因果関係がある」ではありません!

4.6 演習 4-4: ヒートマップで多変量データを見る

```
# 演習 4-4: 国×指標のヒートマップ
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np


# サンプルデータ(主要国の各種ガバナンス指標)
data = {
    '政治安定': [85, 90, 30, 25, 75, 80, 45, 70],
    '法の支配': [88, 92, 35, 30, 80, 85, 40, 75],
    '腐敗管理': [82, 89, 28, 22, 78, 83, 38, 72],
    '規制品質': [86, 91, 32, 27, 79, 84, 42, 73],
    '政府効率': [84, 88, 33, 25, 76, 82, 41, 74],
    '言論自由': [80, 90, 20, 15, 85, 88, 35, 70]
}
```

```
countries = ['Japan', 'Germany', 'Russia', 'N.Korea', 'France', 'UK', 'China',
'Italy']
df = pd.DataFrame(data, index=countries)


# ヒートマップを描く
plt.figure(figsize=(10, 6))
sns.heatmap(df, annot=True, cmap='RdYlGn', center=50,
            cbar_kws={'label': 'Score (0-100)'},
            linewidths=0.5)
plt.title('Country Governance Indicators', fontsize=14)
plt.show()


print("\n 各国の総合スコア(高いほど良好):")
print(df.mean(axis=1).sort_values(ascending=False).round(1))
```

4.7 第 4 回のまとめ

- seaborn は matplotlib の上に作られた美しいグラフライブラリ
- sns.boxplot() で箱ひげ図、sns.histplot() でヒストグラム
- sns.regplot() で散布図と回帰直線を同時に描ける
- sns.heatmap() で多変量データを一目で比較できる
- 相関係数は corr() メソッドで簡単に計算できる

第 4 回 実行結果例

※ グラフ内のラベルは、教材コードの `plt.rcParams['font.family'] = 'DejaVu Sans'` 設定により英語で表示されます(DejaVu Sans は日本語非対応のため)。日本語ラベルにしたい場合は、Google Colab のセル冒頭で `!apt-get -y install fonts-noto-cjk` 等で日本語フォントを導入し、`font.family` を変更してください。

▼ Exercise 4-1 (Boxplot)

```
【地域別 統計量】
              count  mean   std   min   25%   50%   75%   max
region
Africa         15.0  5.06  1.69  1.47  3.86  5.65  6.54  7.11
Asia           15.0  3.52  1.49  0.63  2.80  3.29  4.39  5.87
Europe         15.0  1.61  0.78  0.58  1.04  1.46  2.09  3.47
Middle East    15.0  5.46  2.04  2.08  3.72  5.77  6.41  9.70
```

▼ Exercise 4-2 (Histogram)

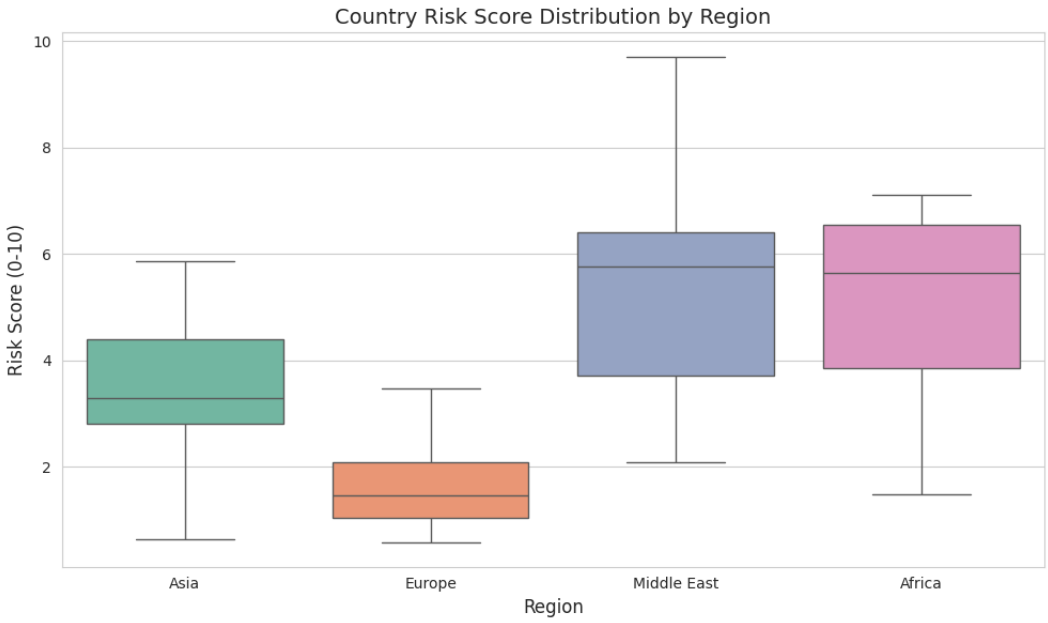
```
平均値: 139 件
中央値: 22 件
差: 117 件
```

▼ Exercise 4-3 (Scatter+Regression)

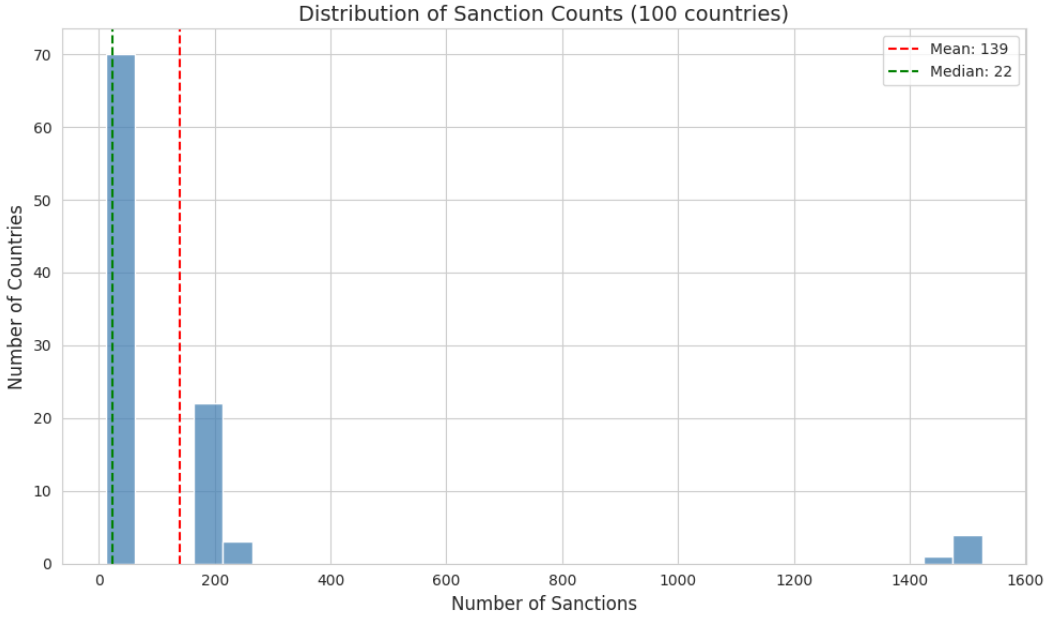
```
相関係数: -0.649
解釈: 強い負の相関
```

▼ Exercise 4-4 (Heatmap)

```
各国の総合スコア(高いほど良好):
Germany    90.0
Japan      84.2
UK         83.7
France     78.8
Italy      72.3
China      40.2
Russia     29.7
N.Korea    24.0
dtype: float64
```

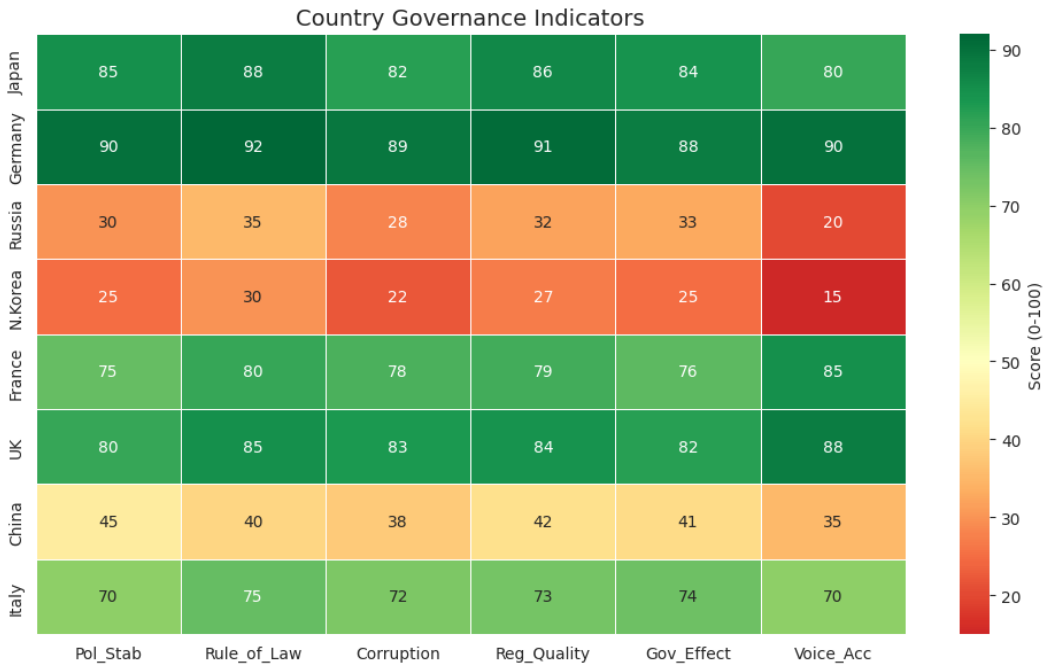


演習 4-1 箱ひげ図: 地域別カントリーリスクの分布



演習 4-2 ヒストグラム: 制裁件数の分布(平均・中央値の縦線付き)

演習 4-3 散布図 + 回帰直線: カントリーリスクと貿易量



演習 4-4 ヒートマップ: 各国ガバナンス指標

第 5 回 記述統計 — 平均・分散・相関を Python で

所要時間: 90 分

5.1 記述統計とは

記述統計(Descriptive Statistics)とは、データの特徴を数値で要約する手法です。Excel の「データ分析」アドインで「基本統計量」を出すのと同じことを、Python でより簡単・自動的に行います。

5.2 主な統計量と Python での計算

統計量	意味	Python コード	Excel 関数
平均	値の合計÷個数	df['col'].mean()	=AVERAGE()
中央値	並べた真ん中の値	df['col'].median()	=MEDIAN()
標準偏差	ばらつきの大きさ	df['col'].std()	=STDEV()
分散	標準偏差の二乗	df['col'].var()	=VAR()

統計量	意味	Python コード	Excel 関数
最大値	一番大きい値	df['col'].max()	=MAX()
最小値	一番小さい値	df['col'].min()	=MIN()
第 1 四分位数	下位 25%の値	df['col'].quantile(0.25)	=QUARTILE(,1)
件数	値の個数	df['col'].count()	=COUNT()
相関係数	2 つの変数の関係性	df['a'].corr(df['b'])	=CORREL()

5.3 演習 5-1: describe()ですべての統計量を一発で見る

pandas の最も便利な機能の 1 つが、df.describe() です。Excel で「データ分析→基本統計量」と何度もクリックしていた作業が、たった 1 行で完了します。

```
# 演習 5-1: 記述統計を一発で取得する
import pandas as pd
import numpy as np


# サンプルデータ
np.random.seed(42)
df = pd.DataFrame({
    'country': ['Japan', 'USA', 'Germany', 'Russia', 'China',
                'India', 'Brazil', 'France', 'UK', 'Italy'],
    'risk_score': np.random.uniform(1, 8, 10),
    'export_volume': np.random.uniform(1000, 10000, 10),
    'sanction_count': np.random.randint(0, 500, 10)
})


# describe() で全部表示
print("【記述統計の自動計算】")
print(df.describe().round(2))


# 個別に取り出す
print("\n【個別の値】")
print(f"リスクスコアの平均: {df['risk_score'].mean():.2f}")
print(f"リスクスコアの中央値: {df['risk_score'].median():.2f}")
print(f"リスクスコアの標準偏差: {df['risk_score'].std():.2f}")
print(f"輸出量の最大値: {df['export_volume'].max():.0f}")
print(f"制裁件数の合計: {df['sanction_count'].sum()}")
```

5.4 演習 5-2: 相関行列で複数の変数の関係を見る

```
# 演習 5-2: 相関行列の計算と可視化
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np


# サンプルデータ(各国の各種指標)
np.random.seed(42)
n = 50
risk = np.random.uniform(1, 8, n)


df = pd.DataFrame({
    'risk_score': risk,
    # リスクと逆相関する貿易量
    'trade_volume': 10000 - 1000 * risk + np.random.normal(0, 1500, n),
    # リスクと正相関する制裁件数
    'sanction_count': risk * 50 + np.random.normal(0, 50, n),
    # リスクと相関する政治安定指標(0-100)
    'political_stability': 100 - 10 * risk + np.random.normal(0, 5, n),
    # 独立した変数(GDP)
    'gdp_per_capita': np.random.uniform(5000, 60000, n)
})


# 相関行列を計算
correlation_matrix = df.corr()
print("【相関行列】")
print(correlation_matrix.round(3))


# 相関行列をヒートマップで可視化
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0,
            vmin=-1, vmax=1, square=True, linewidths=1)
plt.title('Correlation Matrix of Country Indicators', fontsize=14)
plt.tight_layout()
plt.show()
```

5.5 第 5 回のまとめ

- describe() で記述統計を 1 行で計算できる
- .mean(), .median(), .std() などの個別メソッドも便利
- 相関係数は corr() で計算でき、ヒートマップで可視化すると直感的
- 相関と因果は別物 — 必ず文脈と背景知識で解釈する

第 5 回 実行結果例

▼ Exercise 5-1 (describe)

```
【記述統計の自動計算】
       risk_score  export_volume  sanction_count
count       10.00          10.00           10.00
mean         4.64        4557.41          287.50
std          2.21        2721.34          154.66
min          1.41        1185.26           58.00
25%          2.47        2715.74          177.25
50%          5.20        3679.62          229.50
75%          6.08        5513.98          445.00
max          7.66        9729.19          475.00

【個別の値】
リスクスコアの平均: 4.64
リスクスコアの中央値: 5.20
リスクスコアの標準偏差: 2.21
輸出量の最大値: 9729
制裁件数の合計: 2875
```

▼ Exercise 5-2 (Correlation matrix)

```
【相関行列】
                     risk_score  ...  gdp_per_capita
risk_score                1.000  ...          -0.174
trade_volume             -0.839  ...           0.083
sanction_count            0.921  ...          -0.175
political_stability      -0.974  ...           0.198
gdp_per_capita           -0.174  ...           1.000

[5 rows x 5 columns]
```

演習 5-2 相関行列のヒートマップ

第 6 回 データ集計と前半編まとめ

所要時間: 120 分

6.1 groupby による集計の徹底活用

ピボットテーブルが Excel の花形機能であるように、groupby() は pandas の花形機能です。

```
# 演習 6-1: groupby の様々な使い方
import pandas as pd
import numpy as np


# サンプルデータ(架空の取引履歴)
np.random.seed(42)
df = pd.DataFrame({
    'date': pd.date_range('2024-01-01', periods=200, freq='D'),
    'country': np.random.choice(['Japan', 'USA', 'Germany', 'China', 'Russia'],
```

```
200),
    'category': np.random.choice(['電子部品', '機械', '化学品', '繊維'], 200),
    'amount': np.random.uniform(100, 10000, 200).round(0)
})


# 1. 国別の取引額合計
print("【国別 取引額合計】")
print(df.groupby('country')['amount'].sum().sort_values(ascending=False))


# 2. 国別の取引件数と平均額(複数の集計を同時に)
print("\n【国別 件数・平均・合計】")
result = df.groupby('country')['amount'].agg(['count', 'mean', 'sum']).round(0)
print(result)


# 3. 国×カテゴリのクロス集計(Excel のピボットテーブルそのもの)
print("\n【国×カテゴリ クロス集計】")
crosstab = pd.crosstab(df['country'], df['category'], values=df['amount'],
aggfunc='sum')
print(crosstab.round(0))


# 4. 月別集計
df['month'] = df['date'].dt.to_period('M')
print("\n【月別取引額】")
print(df.groupby('month')['amount'].sum())
```

6.2 演習: 前半編まとめプロジェクト

第 1 回〜第 5 回で学んだ内容を統合した実践演習です。実在の公開データ(CISA KEV)を使って、一連の分析を行います。

```
# 演習 6-2: CISA KEV データの総合分析
# 第 1〜5 回のすべてを使った実践プロジェクト


import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt


# 1. CISA KEV を Web から読み込む(第 2 回)
url =
"https://www.cisa.gov/sites/default/files/csv/known_exploited_vulnerabilities.csv"
df = pd.read_csv(url)


# 2. データの概要を把握する(第 2 回)
print(f"=== ステップ 1: データの概要 ===")
print(f"行数: {df.shape[0]}, 列数: {df.shape[1]}")
print(f"列名: {list(df.columns)}")
```

```
# 3. 必要な列だけ抽出して、欠損を処理(第 3 回)
print(f"\n=== ステップ 2: データクリーニング ===")
df_clean = df[['cveID', 'vendorProject', 'product', 'vulnerabilityName',
               'dateAdded', 'shortDescription', 'requiredAction',
               'dueDate', 'knownRansomwareCampaignUse']].copy()
df_clean['dateAdded'] = pd.to_datetime(df_clean['dateAdded'])
df_clean['year'] = df_clean['dateAdded'].dt.year
print(f"クリーニング後: {df_clean.shape[0]}行")


# 4. 集計分析(第 6 回)
print(f"\n=== ステップ 3: ベンダー別集計 ===")
vendor_count =
df_clean.groupby('vendorProject').size().sort_values(ascending=False)
print(vendor_count.head(10))


# 5. 時系列の集計
print(f"\n=== ステップ 4: 年別 新規登録件数 ===")
yearly = df_clean.groupby('year').size()
print(yearly)


# 6. ランサムウェア利用された脆弱性の割合
ransomware_count =
df_clean['knownRansomwareCampaignUse'].str.lower().eq('known').sum()
total = len(df_clean)
print(f"\n=== ステップ 5: ランサムウェア活用率 ===")
print(f"ランサムウェア利用が確認された脆弱性: {ransomware_count}件
({ransomware_count/total*100:.1f}%)")


# 7. 可視化(第 4 回)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))


# 左: ベンダー別 Top10
top_vendors = vendor_count.head(10)
sns.barplot(x=top_vendors.values, y=top_vendors.index, ax=axes[0],
palette='viridis')
axes[0].set_title('Top 10 Vendors by Vulnerability Count')
axes[0].set_xlabel('Number of Vulnerabilities')


# 右: 年別の推移
sns.lineplot(x=yearly.index, y=yearly.values, marker='o', ax=axes[1])
axes[1].set_title('Yearly New Vulnerability Additions')
axes[1].set_xlabel('Year')
axes[1].set_ylabel('Count')
axes[1].set_xticks(yearly.index)


plt.tight_layout()
plt.show()
```

これで前半編の知識が定着しました

上の演習で、あなたは次のすべてを実践しました。

・第 1 回: import 文、変数、print()

・第 2 回: pandas、Web からのデータ読み込み、DataFrame 操作

・第 3 回: 列の選択、欠損処理、データ型変換

・第 4 回: seaborn による可視化(barplot, lineplot)

・第 5 回: 集計とサイズ算出

・第 6 回: groupby による多角的な集計

おめでとうございます。あなたはもう「データ分析者」の入口に立っています。

6.3 第 6 回のまとめ

- groupby はピボットテーブル相当の集計機能
- .agg() で複数の集計を一度に実行できる
- pd.crosstab() でクロス集計表が作れる
- 前半編の総合演習で、データ分析の流れを体得

第 6 回 実行結果例

⚠ 以下の実行結果は、架空データを用いた結果です。受講生の皆様はご自分で実際にウェブサイトからオープンデータを取得した上で、pandas を実行してください。

▼ Exercise 6-1 (groupby)

```
【国別 取引額合計】
country
Japan      246610.0
China      241492.0
Germany    191543.0
Russia     162197.0
USA        159850.0
Name: amount, dtype: float64

【国別 件数・平均・合計】
         count    mean       sum
country
China       49  4928.0  241492.0
```

```
Germany      37  5177.0  191543.0
Japan        43  5735.0  246610.0
Russia       36  4505.0  162197.0
USA          35  4567.0  159850.0

【国×カテゴリ クロス集計】
category       化学品       機械       繊維      電子部品
country
China     83741.0  34348.0  62184.0  61219.0
Germany   38663.0  24806.0  68630.0  59444.0
Japan     44463.0  54318.0  79154.0  68675.0
Russia    40170.0  58610.0  42794.0  20623.0
USA       33858.0  40823.0  46617.0  38552.0

【月別取引額】
month
2024-01    127305.0
2024-02    129531.0
2024-03    143324.0
2024-04    166635.0
2024-05    161686.0
2024-06    174246.0
2024-07     98965.0
Freq: M, Name: amount, dtype: float64
```

▼ Exercise 6-2 (Integrated KEV analysis with sample data)

▼ ステップ 1: データの概要

```
行数: 300, 列数: 9
列名: ['cveID', 'vendorProject', 'product', 'vulnerabilityName', 'dateAdded',
'shortDescription', 'requiredAction', 'dueDate', 'knownRansomwareCampaignUse']
```

▼ ステップ 2: データクリーニング

```
クリーニング後: 300 行
```

▼ ステップ 3: ベンダー別集計

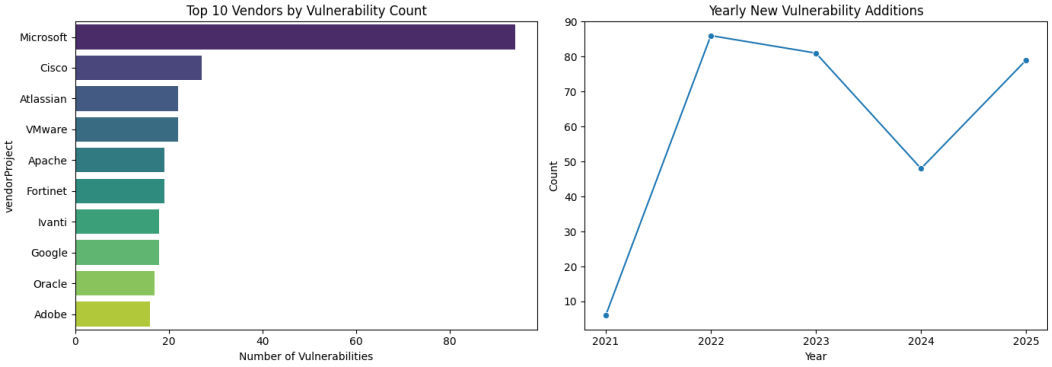
```
vendorProject
Microsoft    94
Cisco        27
Atlassian    22
VMware       22
Apache       19
Fortinet     19
Ivanti       18
Google       18
Oracle       17
Adobe        16
dtype: int64
```

▼ ステップ 4: 年別 新規登録件数

```
year
2021     6
2022    86
2023    81
2024    48
2025    79
dtype: int64
```

▼ ステップ 5: ランサムウェア活用率

```
ランサムウェア利用が確認された脆弱性: 77 件 (25.7%)
```



演習 6-2 前半編まとめ: ベンダー別 Top10 と年別追加件数

第 7 回 ネットワーク分析 — networkx で関係性を描く

所要時間: 120 分

7.1 ネットワーク分析とは

ネットワーク分析(Network Analysis)とは、「ものとものの関係」を分析する手法です。

> **ネットワーク分析の応用例**
>
> ・サプライチェーン: 取引先と取引先の関係
>
> ・制裁回避ルート: 制裁対象企業 → 子会社 → ダミー会社の関係
>
> ・SNS: 人と人のフォロー関係
>
> ・サイバー攻撃: 感染端末と攻撃インフラの関係
>
> 本講座では、Python の networkx モジュールを使って
>
> 「制裁対象企業のサプライチェーン関係」を可視化します。

7.2 ネットワークの基本用語

- ノード(node)、頂点: 「もの」(企業、人、コンピューターなど)
- エッジ(edge)、辺: 「ものとものの関係」(取引、所有、フォロー)
- 有向エッジ: 矢印付きの関係(A → B、A から B へ送る)
- 無向エッジ: 矢印なしの関係(A — B、双方向)

7.3 演習 7-1: 簡単なネットワークの構築と可視化

```
# 演習 7-1: 制裁対象企業のサプライチェーン関係
import networkx as nx
import matplotlib.pyplot as plt

# 有向グラフを作成
G = nx.DiGraph()

# ノード(企業)を追加
companies = {
    'SDN_Corp': {'risk': 'high', 'type': '制裁対象'},
    'Front_A': {'risk': 'high', 'type': 'フロント企業'},
    'Front_B': {'risk': 'high', 'type': 'フロント企業'},
    'Subsidiary_C': {'risk': 'medium', 'type': '子会社'},
    'Trader_D': {'risk': 'medium', 'type': '商社'},
    'Trader_E': {'risk': 'low', 'type': '商社'},
    'JP_Co_1': {'risk': 'low', 'type': '日本企業'},
    'JP_Co_2': {'risk': 'low', 'type': '日本企業'},
    'JP_Co_3': {'risk': 'low', 'type': '日本企業'}
}

for company, attrs in companies.items():
    G.add_node(company, **attrs)

# エッジ(取引関係)を追加
relationships = [
    ('SDN_Corp', 'Front_A'),
    ('SDN_Corp', 'Front_B'),
    ('Front_A', 'Subsidiary_C'),
    ('Front_B', 'Trader_D'),
    ('Subsidiary_C', 'Trader_E'),
    ('Trader_D', 'JP_Co_1'),
    ('Trader_D', 'JP_Co_2'),
    ('Trader_E', 'JP_Co_3')
]

G.add_edges_from(relationships)

# 可視化
plt.figure(figsize=(14, 10))
pos = nx.spring_layout(G, seed=42, k=2)

# ノードの色をリスクレベルで分ける
color_map = {'high': '#E74C3C', 'medium': '#F39C12', 'low': '#27AE60'}
node_colors = [color_map[G.nodes[n]['risk']] for n in G.nodes()]

# ノードを描く
```

```
nx.draw_networkx_nodes(G, pos, node_color=node_colors, node_size=2500, alpha=0.9)
# エッジを描く(矢印付き)
nx.draw_networkx_edges(G, pos, edge_color='gray', arrows=True,
                        arrowsize=20, alpha=0.6, width=2)
# ラベルを描く
nx.draw_networkx_labels(G, pos, font_size=10, font_family='sans-serif')


plt.title('Sanction Risk Network: SDN Company Supply Chain', fontsize=14)
plt.axis('off')


# 凡例
import matplotlib.patches as mpatches
high_patch = mpatches.Patch(color='#E74C3C', label='High Risk')
med_patch = mpatches.Patch(color='#F39C12', label='Medium Risk')
low_patch = mpatches.Patch(color='#27AE60', label='Low Risk')
plt.legend(handles=[high_patch, med_patch, low_patch], loc='upper left')


plt.tight_layout()
plt.show()


# ネットワークの基本情報
print(f"ノード数: {G.number_of_nodes()}")
print(f"エッジ数: {G.number_of_edges()}")
```

7.4 演習 7-2: 中心性指標で「重要なノード」を見つける

```
# 演習 7-2: ネットワーク中心性の計算
import networkx as nx
import pandas as pd


# 上の演習のグラフ G を使う(または再構築)


# 各種中心性を計算する
degree_cent = nx.degree_centrality(G)
betweenness_cent = nx.betweenness_centrality(G)
closeness_cent = nx.closeness_centrality(G)
pagerank = nx.pagerank(G)


# 結果を表にまとめる
metrics = pd.DataFrame({
    'company': list(G.nodes()),
    'degree': [degree_cent[n] for n in G.nodes()],
    'betweenness': [betweenness_cent[n] for n in G.nodes()],
    'closeness': [closeness_cent[n] for n in G.nodes()],
    'pagerank': [pagerank[n] for n in G.nodes()]
}).round(3)


# PageRank で並べ替え
```

```
metrics = metrics.sort_values('pagerank', ascending=False)


print("【ネットワーク中心性指標】")
print(metrics.to_string(index=False))


print("\n【解釈】")
print("・degree(次数): 何社と取引しているか。多いほどネットワークでの存在感が大きい")
print("・betweenness(媒介中心性): 他社の取引の「橋渡し」になる頻度。高いほど制裁回避の要
")
print("・closeness(近接中心性): 全体への距離。情報伝播の速さ")
print("・pagerank: 重要な企業から取引されているほど高くなる")
```

7.5 第 7 回のまとめ

- networkx は Python でネットワークを扱うモジュール
- ノード(企業)とエッジ(取引)で関係を表現
- nx.spring_layout() で自動的にきれいな配置に
- 中心性指標で「重要なノード」を定量的に判定可能

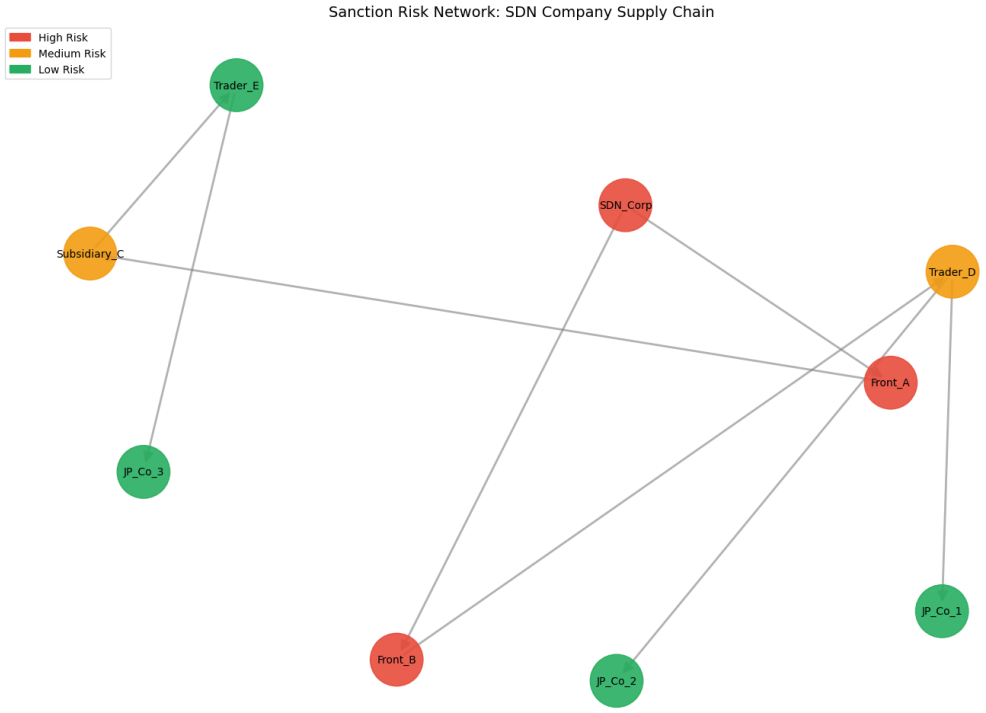
第 7 回 実行結果例

▼ Exercise 7-1 (Network)

```
ノード数: 9
エッジ数: 8
```

▼ Exercise 7-2 (Centrality)

```
【ネットワーク中心性指標】
     company  degree  betweenness  closeness  pagerank
     JP_Co_3   0.125        0.000      0.200     0.187
    Trader_E   0.250        0.054      0.188     0.156
    Trader_D   0.375        0.071      0.167     0.120
Subsidiary_C   0.250        0.071      0.167     0.120
     JP_Co_1   0.125        0.000      0.188     0.105
     JP_Co_2   0.125        0.000      0.188     0.105
     Front_A   0.250        0.054      0.125     0.077
     Front_B   0.250        0.054      0.125     0.077
    SDN_Corp   0.250        0.000      0.000     0.054

【解釈】
・degree(次数): 何社と取引しているか。多いほどネットワークでの存在感が大きい
・betweenness(媒介中心性): 他社の取引の「橋渡し」になる頻度。高いほど制裁回避の要
・closeness(近接中心性): 全体への距離。情報伝播の速さ
・pagerank: 重要な企業から取引されているほど高くなる
```



演習 7-1 サプライチェーンネットワーク図(リスク色分け)

第 8 回 時系列解析 — 地政学イベントの影響を見る

所要時間: 120 分

8.1 時系列データとは

時系列データ(Time Series)とは、時間とともに記録されたデータです。

- 月別・週別の貿易量
- 制裁リストへの新規追加件数
- カントリーリスクスコアの変動
- 脆弱性の追加件数

8.2 演習 8-1: 時系列の可視化と地政学イベントの重ね合わせ

```
# 演習 8-1: 対露輸出の時系列分析
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np


# サンプルデータ(対露輸出の月次データ、ウクライナ侵攻前後)
np.random.seed(42)
dates = pd.date_range('2020-01-01', '2024-12-01', freq='MS')


# 2022 年 2 月以降に大きく減少するパターン
trend = np.linspace(800, 900, len(dates))  # 緩やかな上昇トレンド
seasonal = 50 * np.sin(2 * np.pi * np.arange(len(dates)) / 12)  # 季節変動
noise = np.random.normal(0, 30, len(dates))


# 2022 年 2 月以降は急減
sanction_effect = np.zeros(len(dates))
invasion_idx = (dates >= '2022-02-24').values
sanction_effect[invasion_idx] = -500  # 約 60%減少
# 段階的に少し回復(輸出規制以外の品目)
sanction_effect[invasion_idx] += np.linspace(0, 100, sum(invasion_idx))


export_value = trend + seasonal + noise + sanction_effect


df = pd.DataFrame({
    'date': dates,
```

```
    'export_value': export_value
})

# プロット
plt.figure(figsize=(14, 6))
plt.plot(df['date'], df['export_value'], linewidth=2, color='steelblue')

# 重要な地政学イベントを縦線で表示
events = {
    '2022-02-24': 'Ukraine Invasion',
    '2022-03-01': 'Sanctions Begin',
    '2022-10-07': 'US Semi Restrictions',
    '2023-12-15': 'EU 12th Package'
}
for date, label in events.items():
    plt.axvline(pd.Timestamp(date), color='red', linestyle='--', alpha=0.5)
    plt.text(pd.Timestamp(date), df['export_value'].max(), label,
             rotation=90, fontsize=9, va='top', color='red')

plt.title('Japan Exports to Russia: Monthly Time Series', fontsize=14)
plt.xlabel('Date')
plt.ylabel('Export Value (million USD)')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# 制裁前後の比較
before = df[df['date'] < '2022-02-24']['export_value'].mean()
after = df[df['date'] >= '2022-02-24']['export_value'].mean()
print(f"侵攻前の平均(2020-2022/2): {before:.0f}")
print(f"侵攻後の平均(2022/2 以降): {after:.0f}")
print(f"変化率: {(after - before) / before * 100:.1f}%")
```

8.3 演習 8-2: 移動平均でトレンドを見る

```
# 演習 8-2: 移動平均の計算
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# 上の演習の df を使う(または再構築)

# 3 か月移動平均と 12 か月移動平均
df['MA_3'] = df['export_value'].rolling(window=3).mean()
df['MA_12'] = df['export_value'].rolling(window=12).mean()

# プロット
plt.figure(figsize=(14, 6))
plt.plot(df['date'], df['export_value'], label='Original', alpha=0.5,
```

```
color='steelblue')
plt.plot(df['date'], df['MA_3'], label='3-month MA', linewidth=2, color='orange')
plt.plot(df['date'], df['MA_12'], label='12-month MA', linewidth=2, color='red')
plt.title('Time Series with Moving Averages', fontsize=14)
plt.xlabel('Date')
plt.ylabel('Export Value')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

移動平均(MA)とは

移動平均は、時系列データのノイズを除去してトレンドを見るための手法です。

・3 か月 MA = 直近 3 か月の平均(短期のトレンド)

・12 か月 MA = 直近 12 か月の平均(長期のトレンド)

rolling(window=3).mean() で簡単に計算できます。

8.4 第 8 回のまとめ

- 時系列データは時間順に記録されたデータ
- plt.axvline() で重要なイベントを縦線で重ねられる
- rolling(window=N).mean() で移動平均
- 制裁などのイベントの影響を定量化できる

第 8 回 実行結果例

※ pandas のバージョンによっては、教材記載の `(dates >= '2022-02-24').values` で `AttributeError: 'numpy.ndarray' object has no attribute 'values'` というエラーが出る場合があります。その場合は `np.asarray(dates >= '2022-02-24')` に置き換えてください(本書の実行結果はこの修正後のコードで取得しています)。

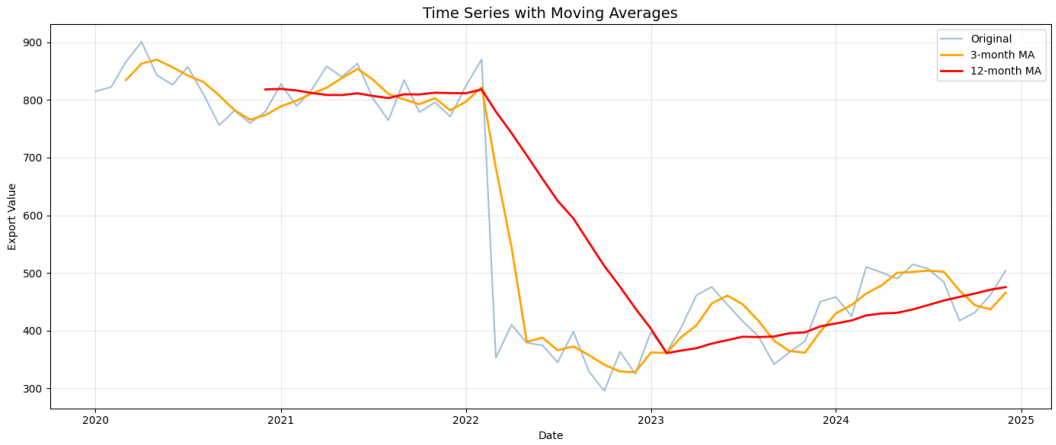
▼ Exercise 8-1 (Time series)

```
侵攻前の平均(2020-2022/2): 818
侵攻後の平均(2022/2 以降): 417
変化率: -49.0%
```

▼ Exercise 8-2 (Moving Average)

```
グラフを生成しました(コードに print 文はないため出力なし)
```

演習 8-1 時系列+地政学イベントの縦線



演習 8-2 移動平均(3 か月・12 か月)

第 9 回 多変量解析 — カントリーリスクの複合評価

所要時間: 120 分

9.1 多変量解析とは

多変量解析(Multivariate Analysis)とは、複数の変数(特徴)を同時に分析する手法です。

カントリーリスク評価では、政治・経済・法律・サイバーなど多次元の指標を「総合的に」見る必要があります。

9.2 演習 9-1: 主成分分析(PCA)で 2 次元に圧縮する

```
# 演習 9-1: PCA で多次元データを 2 次元に圧縮
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import numpy as np


# サンプルデータ(各国の 6 次元のガバナンス指標)
np.random.seed(42)
data = {
    '政治安定': [85, 90, 30, 25, 75, 80, 45, 70, 55, 65],
    '法の支配': [88, 92, 35, 30, 80, 85, 40, 75, 50, 70],
    '腐敗管理': [82, 89, 28, 22, 78, 83, 38, 72, 45, 68],
    '規制品質': [86, 91, 32, 27, 79, 84, 42, 73, 52, 67],
    '政府効率': [84, 88, 33, 25, 76, 82, 41, 74, 48, 66],
    '言論自由': [80, 90, 20, 15, 85, 88, 35, 70, 60, 65]
}
countries = ['Japan', 'Germany', 'Russia', 'N.Korea', 'France',
             'UK', 'China', 'Italy', 'Mexico', 'India']
df = pd.DataFrame(data, index=countries)


# ステップ 1: データの標準化
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df)


# ステップ 2: PCA で 2 次元に圧縮
pca = PCA(n_components=2)
df_pca = pca.fit_transform(df_scaled)


# 結果を DataFrame にまとめる
```

```
pca_df = pd.DataFrame(df_pca, columns=['PC1', 'PC2'], index=countries)
print("【PCA の結果】")
print(pca_df.round(2))
print(f"\n 第 1 主成分の寄与率: {pca.explained_variance_ratio_[0]*100:.1f}%")
print(f"第 2 主成分の寄与率: {pca.explained_variance_ratio_[1]*100:.1f}%")
print(f"合計: {sum(pca.explained_variance_ratio_)*100:.1f}%")


# 可視化
plt.figure(figsize=(12, 8))
plt.scatter(pca_df['PC1'], pca_df['PC2'], s=300, alpha=0.6, c='steelblue',
edgecolors='black')


for country, (pc1, pc2) in pca_df.iterrows():
    plt.annotate(country, (pc1, pc2), fontsize=11, ha='center', va='center')


plt.axhline(0, color='gray', linewidth=0.5)
plt.axvline(0, color='gray', linewidth=0.5)
plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%)', fontsize=12)
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)', fontsize=12)
plt.title('Country Profile in 2D Space (PCA)', fontsize=14)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

🎯 PCA の解釈

PCA は「6 次元のデータを、できるだけ情報を失わずに 2 次元に圧縮する」手法です。

上のグラフでは、

・近くの国 = 似たガバナンス・プロファイル

・遠くの国 = 異なるプロファイル

民主主義国(Japan, Germany, France, UK)は近くに、

権威主義国(Russia, N.Korea, China)は別の位置に集まる傾向が見られます。

9.3 演習 9-2: クラスタリングで自動的にグループ分け

```
# 演習 9-2: K-means クラスタリングで国を自動分類
from sklearn.cluster import KMeans
import seaborn as sns


# 上の演習の pca_df と df_scaled を使う
```

```
# K-means(k=3 でクラスタリング)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = kmeans.fit_predict(df_scaled)


# 結果を PCA 図にプロット
pca_df['cluster'] = clusters


plt.figure(figsize=(12, 8))
colors = ['#E74C3C', '#3498DB', '#27AE60']
for c in range(3):
    cluster_data = pca_df[pca_df['cluster'] == c]
    plt.scatter(cluster_data['PC1'], cluster_data['PC2'],
                s=300, alpha=0.7, c=colors[c],
                edgecolors='black', label=f'Cluster {c}')


for country, row in pca_df.iterrows():
    plt.annotate(country, (row['PC1'], row['PC2']),
                 fontsize=10, ha='center', va='center')


plt.xlabel('PC1', fontsize=12)
plt.ylabel('PC2', fontsize=12)
plt.title('Country Clusters by Governance Profile', fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()


# 各クラスターの典型的な特徴を表示
print("\n【クラスター別の国一覧】")
for c in range(3):
    cluster_countries = pca_df[pca_df['cluster'] == c].index.tolist()
    print(f"Cluster {c}: {cluster_countries}")
```

9.4 第 9 回のまとめ

- PCA(主成分分析)で多次元データを 2 次元に圧縮
- StandardScaler でデータを正規化してから分析
- K-means で自動的にグループ分けが可能
- scikit-learn は機械学習の代表的モジュール

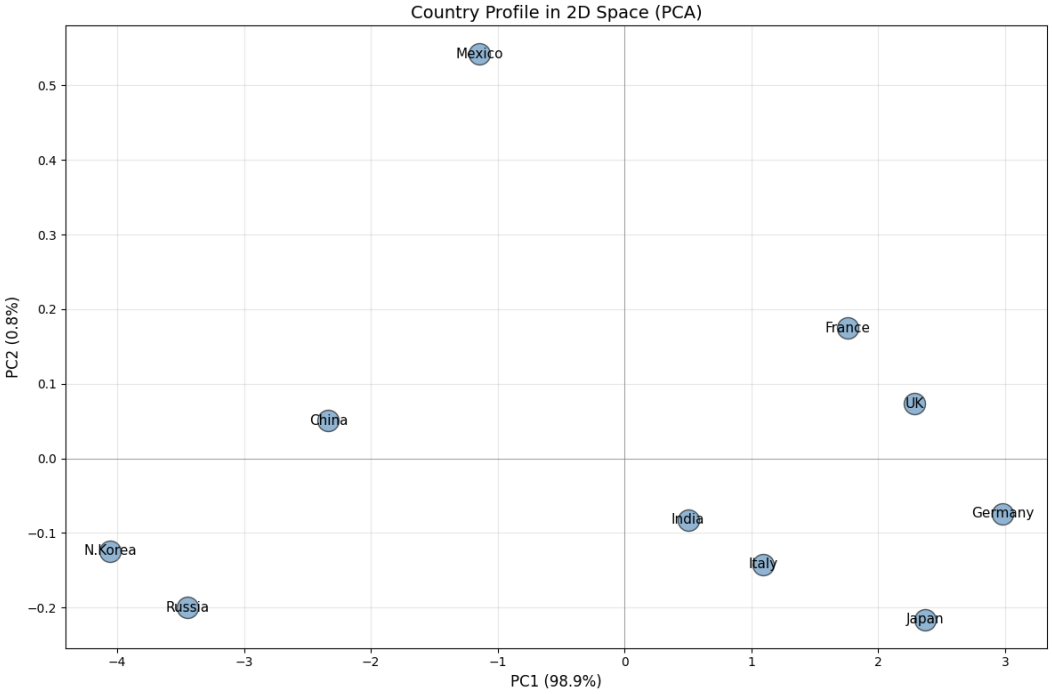
第 9 回 実行結果例

▼ Exercise 9-1 (PCA)

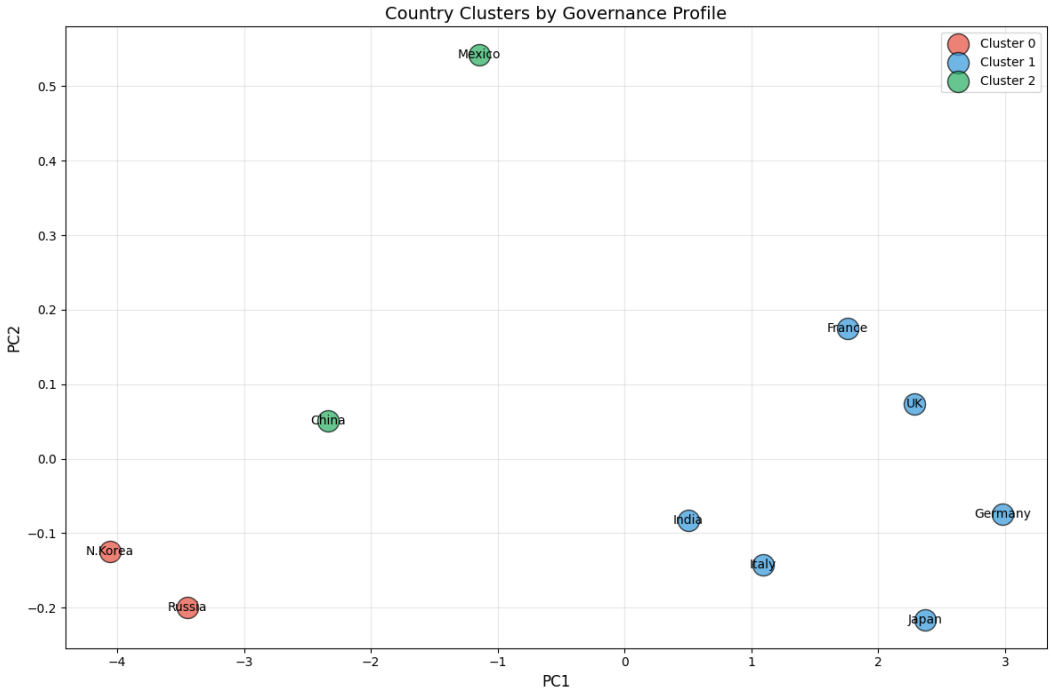
```
【PCA の結果】
          PC1   PC2
Japan    2.37 -0.22
Germany  2.98 -0.07
Russia  -3.45 -0.20
N.Korea -4.06 -0.12
France   1.76  0.17
UK       2.28  0.07
China   -2.34  0.05
Italy    1.09 -0.14
Mexico  -1.15  0.54
India    0.50 -0.08

第 1 主成分の寄与率: 98.9%
第 2 主成分の寄与率: 0.8%
合計: 99.7%
```

▼ Exercise 9-2 (K-means)

```
【クラスター別の国一覧】
Cluster 0: ['Russia', 'N.Korea']
Cluster 1: ['Japan', 'Germany', 'France', 'UK', 'Italy', 'India']
Cluster 2: ['China', 'Mexico']
```



演習 9-1 PCA による 2 次元プロット(各国のガバナンス・プロファイル)



演習 9-2 K-means クラスタリングの結果

第 10 回 サイバーセキュリティデータの分析

所要時間: 120 分

10.1 サイバーセキュリティと経済安全保障の交差

経済安全保障の重要要素として、サイバー脅威があります。本回では、米国 CISA が公開している実データを使って分析を行います。

10.2 演習 10-1: CISA KEV の総合分析

```
# 演習 10-1: CISA KEV データの徹底分析
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt


# CISA KEV を読み込む
url =
"https://www.cisa.gov/sites/default/files/csv/known_exploited_vulnerabilities.csv"
df = pd.read_csv(url)


# 日付列を変換
```

```
df['dateAdded'] = pd.to_datetime(df['dateAdded'])
df['year_month'] = df['dateAdded'].dt.to_period('M')


# 1. ベンダー別 Top 15
top_vendors =
df.groupby('vendorProject').size().sort_values(ascending=False).head(15)


# 2. 月別の追加件数推移
monthly = df.groupby('year_month').size()


# 3. ランサムウェア利用フラグの集計
ransomware = df['knownRansomwareCampaignUse'].fillna('Unknown').value_counts()


# 4 つのグラフを並べる
fig, axes = plt.subplots(2, 2, figsize=(16, 12))


# 左上: ベンダー別 Top15
sns.barplot(x=top_vendors.values, y=top_vendors.index, ax=axes[0, 0],
palette='viridis')
axes[0, 0].set_title('Top 15 Vendors by Vulnerability Count')
axes[0, 0].set_xlabel('Count')


# 右上: 月別推移
axes[0, 1].plot(monthly.index.astype(str), monthly.values, marker='o', linewidth=2)
axes[0, 1].set_title('Monthly New Vulnerability Additions')
axes[0, 1].set_xlabel('Month')
axes[0, 1].set_ylabel('Count')
axes[0, 1].tick_params(axis='x', rotation=45)


# 左下: ランサムウェア利用
axes[1, 0].pie(ransomware.values, labels=ransomware.index, autopct='%1.1f%%',
               colors=['#E74C3C', '#27AE60', '#95A5A6'])
axes[1, 0].set_title('Ransomware Campaign Use')


# 右下: 年別件数
yearly = df.groupby(df['dateAdded'].dt.year).size()
sns.barplot(x=yearly.index, y=yearly.values, ax=axes[1, 1], palette='rocket')
axes[1, 1].set_title('Yearly Vulnerabilities Added')
axes[1, 1].set_xlabel('Year')


plt.tight_layout()
plt.show()


# サマリー
print(f"\n=== CISA KEV データサマリー ===")
print(f"総件数: {len(df)}件")
print(f"対象期間: {df['dateAdded'].min().date()} 〜 {df['dateAdded'].max().date()}")
print(f"ベンダー数: {df['vendorProject'].nunique()}社")
```

```
print(f"ランサム利用確認: {(df['knownRansomwareCampaignUse'].str.lower() ==
'known').sum()}件")
```

10.3 第 10 回のまとめ

- 実在の CISA KEV データを Web から直接読み込み可能
- 複数のサブプロットでダッシュボード風の分析が可能
- 脅威の種類・頻度・トレンドを定量的に把握

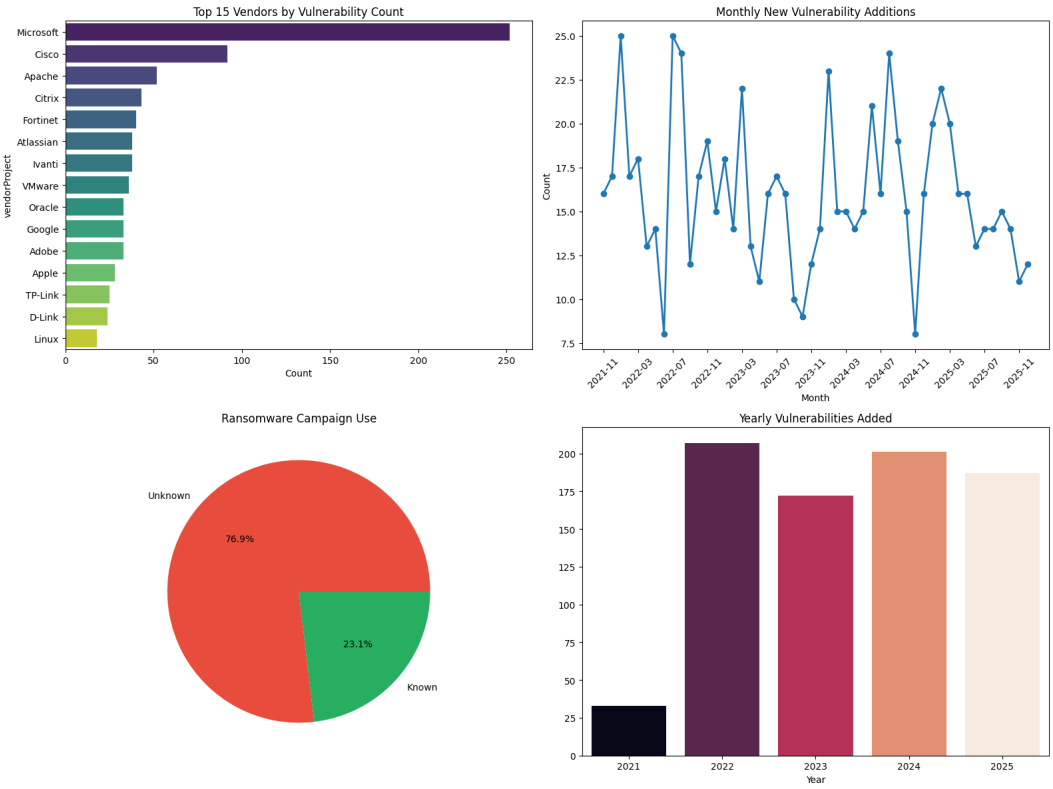
第 10 回 実行結果例

⚠ 以下の実行結果は、架空データを用いた結果です。受講生の皆様はご自分で実際にウェブサイトからオープンデータを取得した上で、pandas を実行してください。

▼ Exercise 10-1 (KEV Dashboard with Sample)

▼ CISA KEV データサマリー(サンプル)

```
総件数: 800 件
対象期間: 2021-11-09 ～ 2025-12-28
ベンダー数: 16 社
ランサム利用確認: 185 件
```



演習 10-1 KEV 総合分析ダッシュボード(4 分割)

第 11 回 統合分析レポート — 複数データを組み合わせる

所要時間: 150 分

11.1 統合分析の意義

実務では、単一のデータソースだけで判断することはまれです。複数のデータを組み合わせて、多角的な評価を行います。

11.2 演習 11-1: 統合リスク評価レポート

```
# 演習 11-1: 複数データソースを統合したリスク評価
# このコードはサンプルデータで動作します
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np


# サンプル: 取引先一覧
np.random.seed(42)
suppliers = pd.DataFrame({
    'supplier_id': [f'S{i:03d}' for i in range(1, 21)],
    'name': [f'Supplier_{i}' for i in range(1, 21)],
    'country': np.random.choice(['JP', 'US', 'CN', 'RU', 'DE', 'KR', 'TW', 'IN'],
20),
    'annual_spend': np.random.uniform(100, 5000, 20).round(0),
    'product_category': np.random.choice(['電子部品', '機械', '化学品', 'ソフトウェア
'], 20)
})


# サンプル: 国別カントリーリスク
country_risk = pd.DataFrame({
    'country': ['JP', 'US', 'CN', 'RU', 'DE', 'KR', 'TW', 'IN'],
    'country_risk_score': [2.1, 3.0, 5.5, 7.5, 2.8, 3.2, 4.0, 4.5],
    'sanctions_active': [False, False, True, True, False, False, False, False]
})


# サンプル: 制裁リスト(架空)
sanction_list = pd.DataFrame({
    'name': ['Supplier_5', 'Supplier_15'],
    'list_type': ['SDN', 'Entity List'],
    'date_added': ['2023-06-15', '2024-01-20']
})
```

```
# === 統合分析 ===
# 1. カントリーリスクを結合
df = suppliers.merge(country_risk, on='country', how='left')

# 2. 制裁リスト一致を確認
df = df.merge(sanction_list, on='name', how='left')
df['sanctioned'] = df['list_type'].notna()

# 3. 統合リスクスコアの計算
df['integrated_risk'] = (
    df['country_risk_score'] * 0.5 +  # カントリーリスク
    df['sanctions_active'].astype(int) * 5 +  # 国家制裁
    df['sanctioned'].astype(int) * 10  # 個別制裁
)

# 4. リスクレベル分類
df['risk_level'] = pd.cut(df['integrated_risk'],
                          bins=[0, 3, 6, 100],
                          labels=['低', '中', '高'])

# 結果表示
print("【統合リスク評価結果】")
print(df[['supplier_id', 'name', 'country', 'annual_spend',
          'integrated_risk', 'risk_level', 'sanctioned']].sort_values(
    'integrated_risk', ascending=False))

# 可視化
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# 左: リスクレベル別取引額
risk_summary = df.groupby('risk_level')['annual_spend'].sum()
axes[0].pie(risk_summary.values, labels=risk_summary.index, autopct='%1.1f%%',
            colors=['#27AE60', '#F39C12', '#E74C3C'])
axes[0].set_title('Annual Spend by Risk Level')

# 右: 国別の取引額とリスクスコア
country_summary = df.groupby('country').agg({
    'annual_spend': 'sum',
    'country_risk_score': 'first'
}).reset_index()
axes[1].scatter(country_summary['country_risk_score'],
                country_summary['annual_spend'],
                s=200, alpha=0.6, c='steelblue', edgecolors='black')
for _, row in country_summary.iterrows():
    axes[1].annotate(row['country'],
                     (row['country_risk_score'], row['annual_spend']),
                     fontsize=10, ha='center')
axes[1].set_xlabel('Country Risk Score')
```

```
axes[1].set_ylabel('Annual Spend (million USD)')
axes[1].set_title('Country Risk vs Annual Spend')
axes[1].grid(True, alpha=0.3)


plt.tight_layout()
plt.show()


# 高リスク取引先のアラート
high_risk = df[df['risk_level'] == '高'].sort_values('integrated_risk',
ascending=False)
if len(high_risk) > 0:
    print("\n=== 🚨 高リスク取引先アラート ===")
    print(high_risk[['supplier_id', 'name', 'country', 'integrated_risk',
                     'sanctioned']].to_string(index=False))
```

11.3 第 11 回のまとめ

- 複数データソースを merge() で結合
- 複合スコアを算出することで多角的なリスク評価が可能
- pd.cut() でスコアをカテゴリ(高・中・低)に分類
- 実務でのリスク評価は、必ず複数指標の組み合わせ

第 11 回 実行結果例

※ 教材コードは `pd.cut(..., labels=['低','中','高'])` で日本語ラベルを使っていますが、グラフ描画時に matplotlib のデフォルトフォント(DejaVu Sans)で文字化け(豆腐表示)するのを防ぐため、本書の実行結果では `['Low', 'Medium', 'High']` に置き換えています。Google Colab で日本語フォントを導入すれば、教材記載のままで日本語ラベルが表示できます。

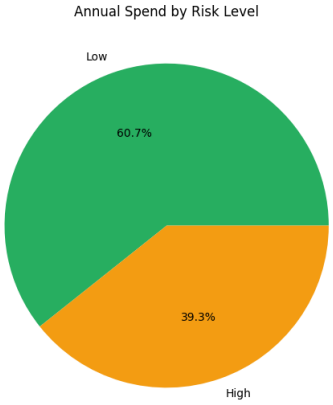
▼ Exercise 11-1 (Integrated Risk Score)

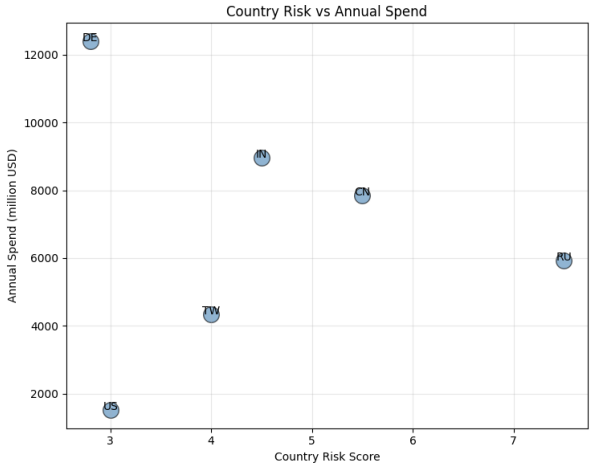
```
【統合リスク評価結果】
   supplier_id         name country  ...  integrated_risk  risk_level sanctioned
4         S005   Supplier_5      CN  ...            17.75        High       True
14        S015  Supplier_15      IN  ...            12.25        High       True
1         S002   Supplier_2      RU  ...             8.75        High      False
16        S017  Supplier_17      RU  ...             8.75        High      False
12        S013  Supplier_13      CN  ...             7.75        High      False
13        S014  Supplier_14      CN  ...             7.75        High      False
19        S020  Supplier_20      CN  ...             7.75        High      False
10        S011  Supplier_11      CN  ...             7.75        High      False
18        S019  Supplier_19      IN  ...             2.25         Low      False
17        S018  Supplier_18      IN  ...             2.25         Low      False
5         S006   Supplier_6      IN  ...             2.25         Low      False
0         S001   Supplier_1      TW  ...             2.00         Low      False
8         S009   Supplier_9      TW  ...             2.00         Low      False
3         S004   Supplier_4      TW  ...             2.00         Low      False
11        S012  Supplier_12      TW  ...             2.00         Low      False
9         S010  Supplier_10      US  ...             1.50         Low      False
2         S003   Supplier_3      DE  ...             1.40         Low      False
7         S008   Supplier_8      DE  ...             1.40         Low      False
6         S007   Supplier_7      DE  ...             1.40         Low      False
15        S016  Supplier_16      DE  ...             1.40         Low      False

[20 rows x 7 columns]
```

▼ Exercise 11-1 High Risk Alert

▼ 🚨 高リスク取引先アラート

```
supplier_id         name country  integrated_risk  sanctioned
       S005   Supplier_5      CN            17.75        True
       S015  Supplier_15      IN            12.25        True
       S017  Supplier_17      RU             8.75       False
       S002   Supplier_2      RU             8.75       False
       S013  Supplier_13      CN             7.75       False
       S011  Supplier_11      CN             7.75       False
       S014  Supplier_14      CN             7.75       False
       S020  Supplier_20      CN             7.75       False
```





演習 11-1 統合リスク評価結果(リスクレベル別取引額・国別散布図)

第 12 回 総合演習と次へのステップ

所要時間: 180 分

12.1 総合演習: あなたの業務に応用する

第 1 回〜第 11 回で学んだすべての技術を使って、自社の業務課題に Python を適用してみましょう。

> **業界別の応用例**
>
> ・製造業: サプライヤーのリスク評価と代替先の優先順位付け
>
> ・金融機関: 与信ポートフォリオの統合リスク分析
>
> ・不動産: 海外物件の地政学リスクスクリーニング
>
> ・商社: 取引先の SDN/Entity List 照合の自動化
>
> ・IT: ベンダーの CVSS スコア統合と対応優先度マトリックス

12.2 修了後の継続学習ロードマップ

ステップ 1: 同じ知識を実務で繰り返し使う

学んだ技術を、毎月の定型業務に組み込んでください。同じ分析を 5 回繰り返せば、自分のものになります。

ステップ 2: 周辺技術を学ぶ

- scikit-learn: 機械学習(分類・回帰・クラスタリング)
- plotly: インタラクティブな可視化
- requests + BeautifulSoup: Web スクレイピング
- SQLAlchemy: データベース連携

ステップ 3: GI 理論などの高度な領域へ

Python でデータ分析ができるようになったあなたは、次の段階として『多様体上のデータ分析』というフロンティアに進めます。これは GI 理論の前段階となる学習領域です。

12.3 講座総括

全 12 回を通じて、あなたは次のすべてを習得しました。

- Python の基本構文(変数、関数、条件分岐、ループ)
- pandas によるデータ読み込み・前処理・集計
- seaborn による多彩な可視化(箱ひげ図・ヒストグラム・散布図・ヒートマップ)
- networkx によるネットワーク分析
- scikit-learn による多変量解析(PCA、クラスタリング)
- 時系列データの取り扱い
- 複数データソースの統合分析

> **最後に**
>
> Excel を 5 年使ってきたあなたが、12 回の学習で Python を使えるようになりました。
>
> これは大きな一歩です。
>
>
> 今後も繰り返し使うことで、Python はあなたの業務の強力な武器になります。
>
> そして、Python は「学んで終わり」ではなく、

> 「学び続けて成長する」道具です。
>
> あなたの実務に役立つことを心から願っています。
>
> Étale Cohomology

付録 A 公開データソース一覧

本講座で使用した、または使用可能な公開データソースを整理します。すべて無料でアクセスできます。

制裁・取引規制データ

リスト名	提供機関	URL
OFAC SDN リスト	米国財務省	ofac.treasury.gov
Entity List	米国商務省 BIS	bis.doc.gov
OpenSanctions	OSS/NGO	opensanctions.org
外務省制裁情報	日本外務省	mofa.go.jp/mofaj/gaiko/sanction/

カントリーリスク・ガバナンス

指標名	提供機関	URL
World Bank WGI	世界銀行	info.worldbank.org/governance/wgi
OECD Country Risk	OECD	oecd.org
CPI(腐敗認識)	Transparency Int.	transparency.org/en/cpi

貿易・輸出統計

データ名	提供機関	URL
UN Comtrade	国連統計部	comtrade.un.org
WTO Statistics	WTO	stats.wto.org
財務省貿易統計	財務省関税局	customs.go.jp/toukei/info/

サイバーセキュリティ

データ名	提供機関	URL
CISA KEV	米国 CISA	cisa.gov/known-exploited-vulnerabilities-catalog
NVD CVE	NIST	nvd.nist.gov
MITRE ATT&CK	MITRE	attack.mitre.org

データ利用上の注意

本教材は教育目的での参照を前提としています。

業務で実際に活用される際は、各データソースの最新の利用規約を必ずご確認ください。

・米国政府公開データ: 自由に利用可能

・World Bank WGI: Creative Commons Attribution 4.0

・OECD Country Risk: 教育目的での参考使用(再配布不可)

・OpenSanctions: 非商用利用は無料、商用利用は別途ライセンス

・財務省貿易統計: 公開統計として一般利用可能

付録 B Python コードクイックリファレンス

基本構文

```
# 変数
x = 10
name = "Étale"


# リスト
companies = ['A 社', 'B 社', 'C 社']


# 辞書
data = {'name': 'A 社', 'risk': 5}


# 条件分岐
if x > 7:
    print("高")
elif x > 4:
    print("中")
else:
    print("低")


# 繰り返し
for company in companies:
    print(company)


# 関数
def calculate_risk(score):
    return score * 2
```

pandas 基本

```
import pandas as pd


# 読み込み
df = pd.read_csv('file.csv')        # CSV
df = pd.read_excel('file.xlsx')      # Excel
df = pd.read_csv(url)                # Web から CSV


# 確認
df.head()        # 最初の 5 行
df.tail()        # 最後の 5 行
df.shape         # (行数, 列数)
df.columns       # 列名
df.info()        # 情報
```

```
df.describe()    # 統計量


# 抽出
df['col']                            # 1 列
df[['col1', 'col2']]                  # 複数列
df[df['col'] > 5]                    # 条件
df[(df['a'] > 5) & (df['b'] < 10)]   # 複数条件


# 並べ替え
df.sort_values('col', ascending=False)


# 集計
df.groupby('col').size()             # 件数
df.groupby('col')['amount'].sum()    # 合計
df.groupby('col').agg(['mean', 'sum']) # 複数


# 結合
df1.merge(df2, on='key', how='left')


# 欠損処理
df.dropna()                          # 削除
df.fillna(0)                         # 埋める
df['col'].fillna(df['col'].mean())   # 平均で埋める
```

seaborn 基本

```
import seaborn as sns
import matplotlib.pyplot as plt


# 箱ひげ図
sns.boxplot(x='category', y='value', data=df)


# ヒストグラム
sns.histplot(data=df, x='value', bins=30)


# 散布図 + 回帰直線
sns.regplot(x='x', y='y', data=df)


# 棒グラフ
sns.barplot(x='category', y='value', data=df)


# ヒートマップ
sns.heatmap(matrix, annot=True, cmap='coolwarm')


# 表示
```

```
plt.show()
```

networkx 基本

```
import networkx as nx


# グラフ作成
G = nx.DiGraph()  # 有向
G = nx.Graph()    # 無向


# ノード追加
G.add_node('A')
G.add_node('B', risk='high')


# エッジ追加
G.add_edge('A', 'B')
G.add_edges_from([('A', 'B'), ('B', 'C')])


# 中心性
nx.degree_centrality(G)
nx.betweenness_centrality(G)
nx.pagerank(G)


# 可視化
nx.draw(G, with_labels=True)
```

付録 C よくあるエラーと対処法

エラー1: ModuleNotFoundError

```
ModuleNotFoundError: No module named 'seaborn'
```

【意味】 そのモジュール(seaborn)が読み込めない

【対処】 セルに次のコードを書いて実行(Colab でも有効)

```
!pip install seaborn
```

エラー2: NameError

```
NameError: name 'df' is not defined
```

【意味】 df という変数が見つからない

【対処】 df を作る前のセル(import 文、データ読み込み)を先に実行する

エラー3: KeyError

```
KeyError: 'country_name'
```

【意味】 'country_name'という列が DataFrame に存在しない

【対処】 df.columns で正しい列名を確認する。スペルミスや大文字小文字の違いに注意

エラー4: IndentationError

```
IndentationError: unexpected indent
```

【意味】 字下げが正しくない

【対処】 字下げを半角スペース 4 個に統一する。Colab が自動で揃えるので、不要な字下げを削除

エラー5: ValueError(日付)

```
ValueError: time data '2024/13/45' does not match format
```

【意味】 日付として解釈できない値がある

【対処】 errors='coerce' を指定すると、変換できない値は NaN になる

```
df['date'] = pd.to_datetime(df['date'], errors='coerce')
```

エラー6: Web 読み込みエラー

```
HTTPError: 404 Not Found
```

【意味】 その URL にファイルがない、またはサーバーが応答しない

【対処】 URL が正しいか確認。ファイルが移動した可能性もあるので、提供機関のサイトで最新 URL を確認する

> **エラーが出ても焦らない**
>
> プログラミングの世界では、エラーは「会話」です。
>
> エラーメッセージは「ここがおかしいよ」と教えてくれているだけです。
>
> 1. エラーメッセージを最初の行から読む
> 2. 何行目で起きたかを確認する
> 3. 上記の対処法を試す
> 4. それでも解決しない場合は、エラーメッセージを Google 検索
>
> ベテランのエンジニアでも、毎日エラーと戦っています。
>
> エラーは「敵」ではなく「先生」です。

おわり

本教材へのご質問・ご意見は、GitHub Issue にてお寄せください。

(C) 2026 Étale Cohomology

License: Creative Commons Attribution 4.0 International (CC BY 4.0)